

Lecture 17: Signal's double ratchet

Announcements

- Weekly reading: Boneh-Shoup, parts of chapter 9 on authenticated encryption
- HW 7 is due Friday 4/4
- Test 2 is next Monday 4/7 in IEC B10 during discussion lab (9:05-9:55 / 10:10-11)

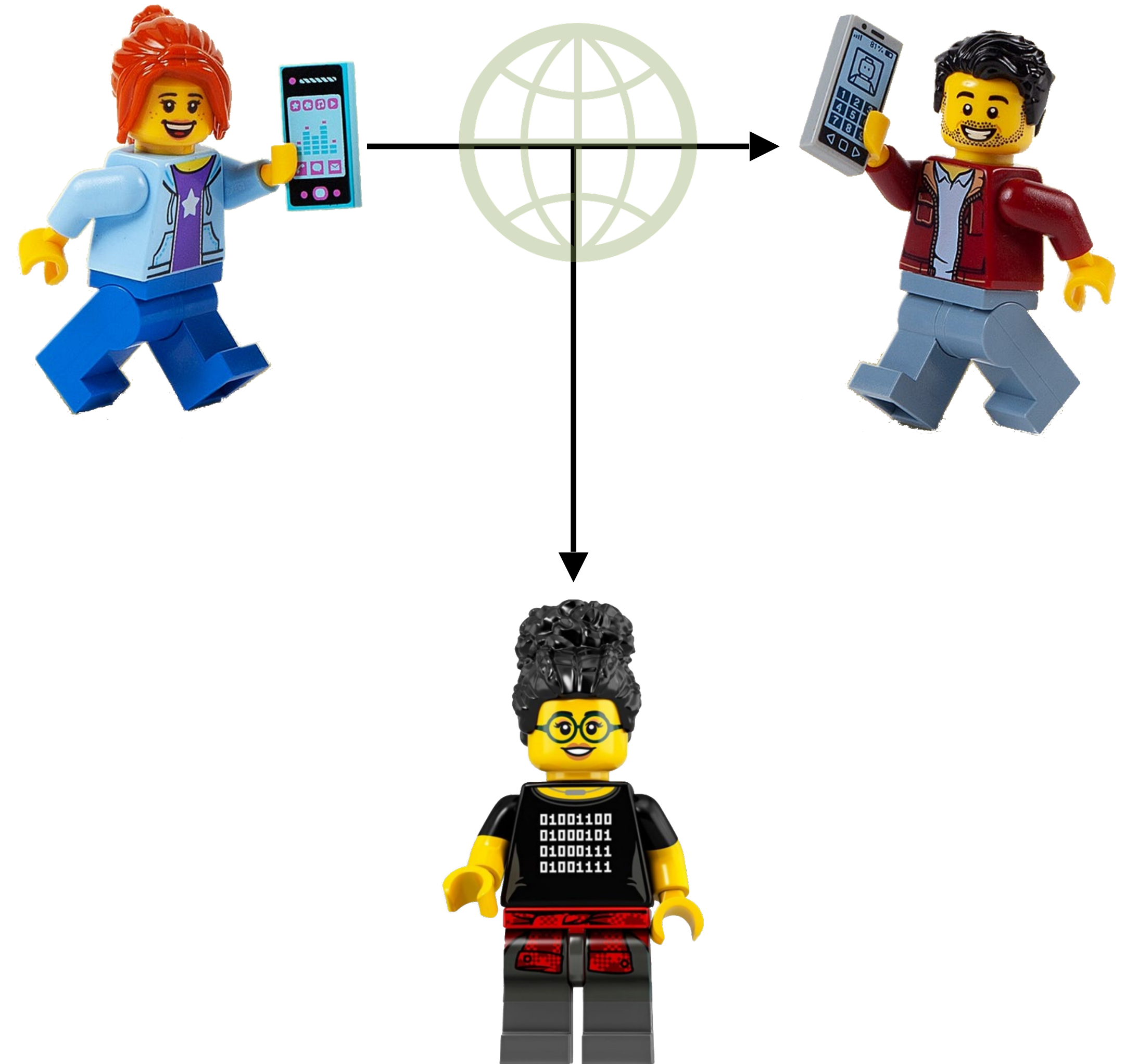
Review: client-server encryption

- Alice wants to talk with a server Bob over the Internet
- They do *not* yet possess a shared secret key
- Our adversary Mallory can read, tamper, add, drop data in transit
 - Mallory is a stand-in for anyone that owns Internet infrastructure



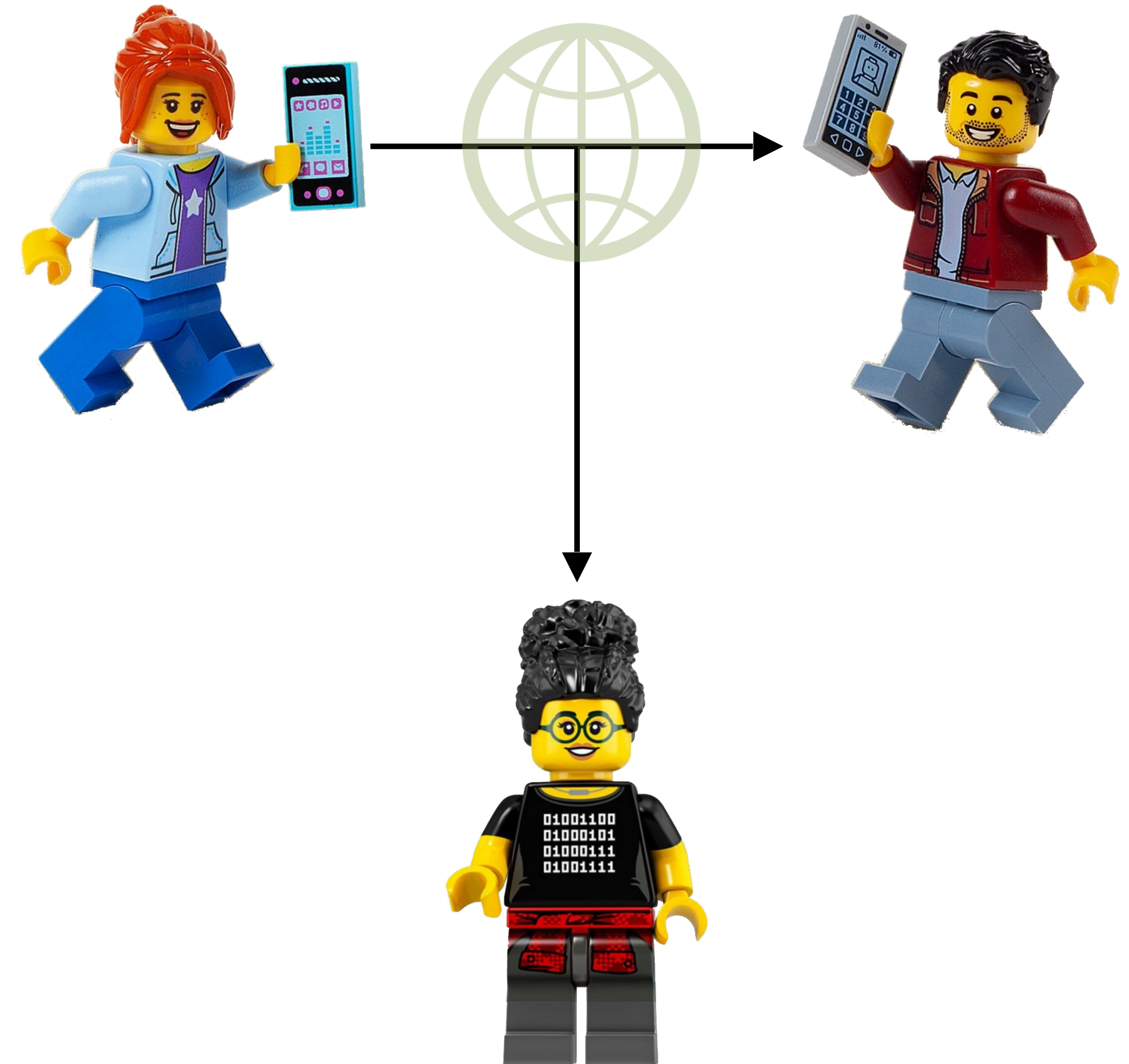
Review: end-to-end encryption

- Now Alice is communicating with Bob's personal laptop/phone
 - They have never met before in person to exchange a key
- Protecting both scenarios involves similar crypto



Objectives

- Protection from network
 - Message confidentiality
 - Sender authenticity + message binding
- Protection from endpoints
 - Secrecy before/after compromise
 - Sender deniability
- Non-goals
 - Hiding metadata (e.g., Alice and Bob's identity, message size)
 - Stopping replay, delay, re-ordering



Objectives

- Protection from network
 - Message confidentiality
 - Sender authenticity + message binding

AuthEnc will protect communication on the network, if Alice and Bob already have a shared key K

- Protection from endpoints
 - Secrecy before/after compromise
 - Sender deniability

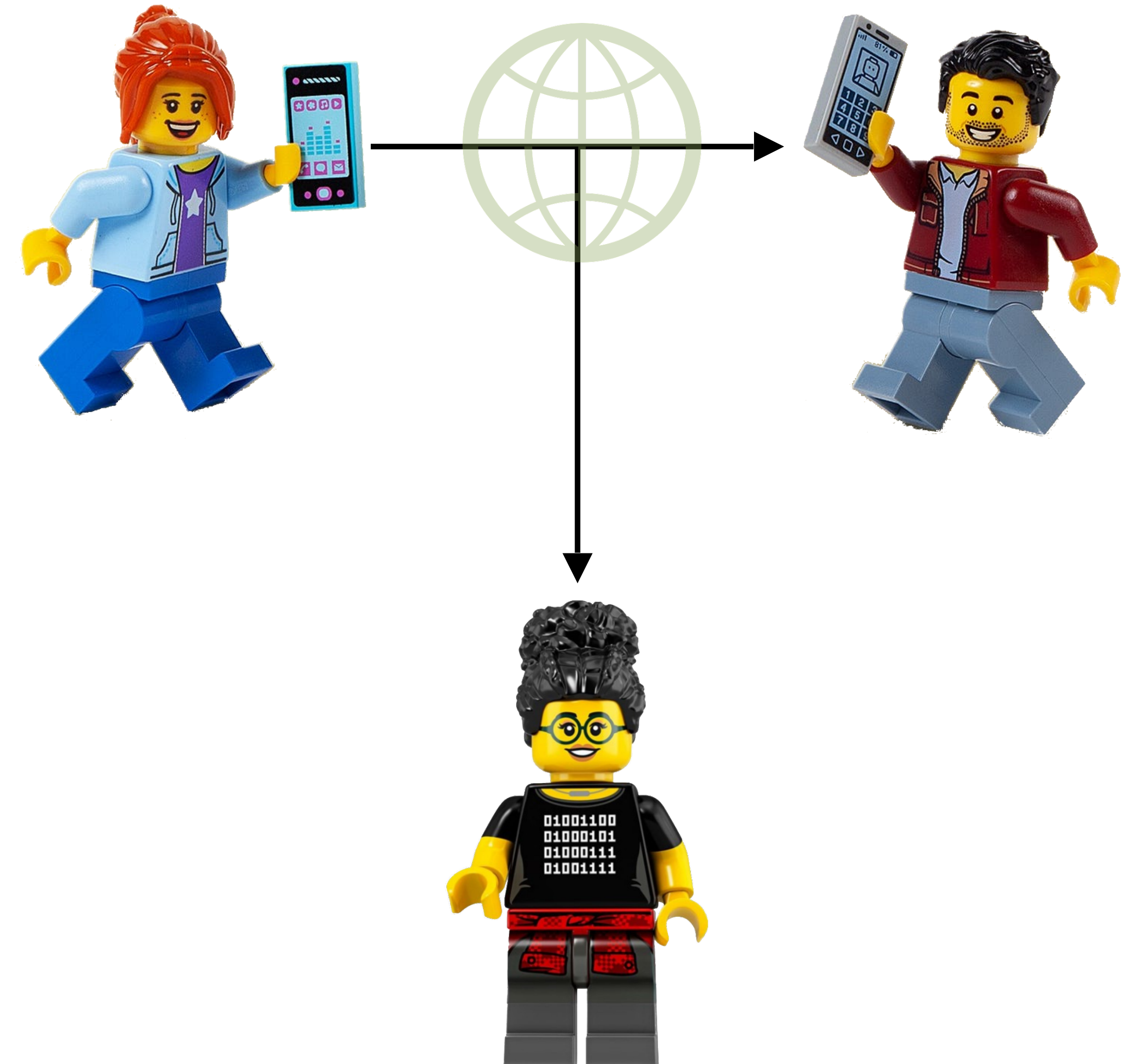
This is new to us...

- Non-goals
 - Hiding metadata (e.g., Alice and Bob's identity, message size)
 - Stopping replay, delay, re-ordering

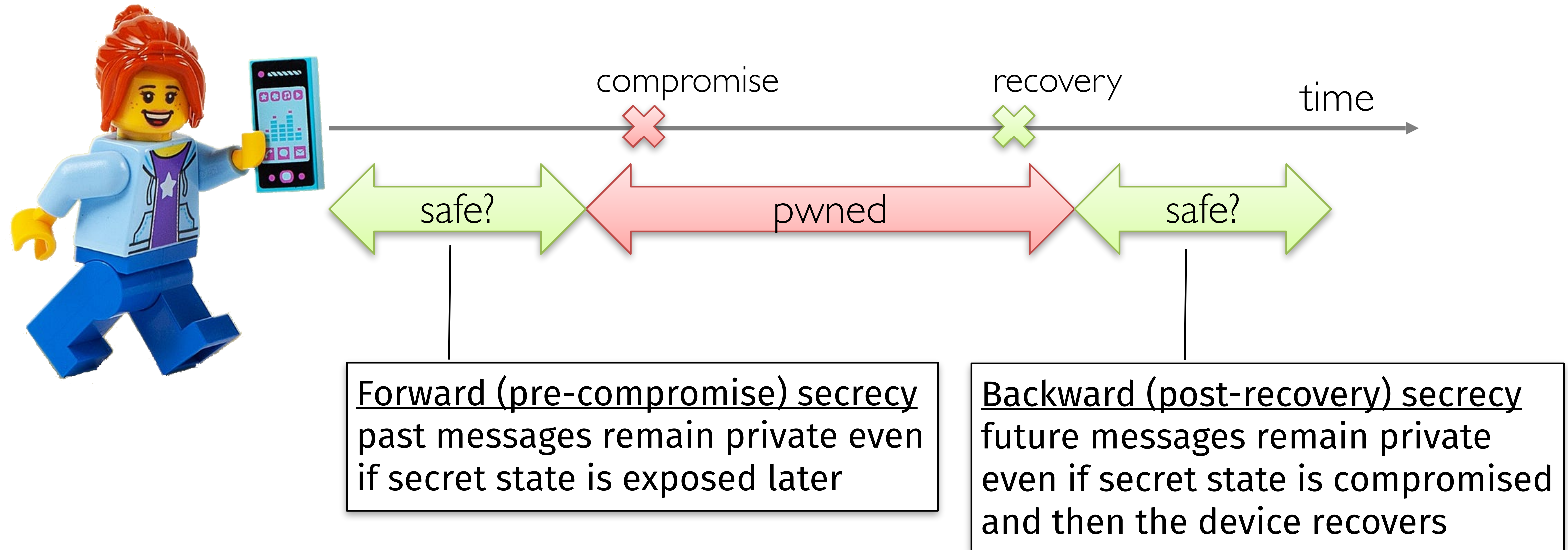
17.1 Forward + backward secrecy and deniability

Overview of Signal

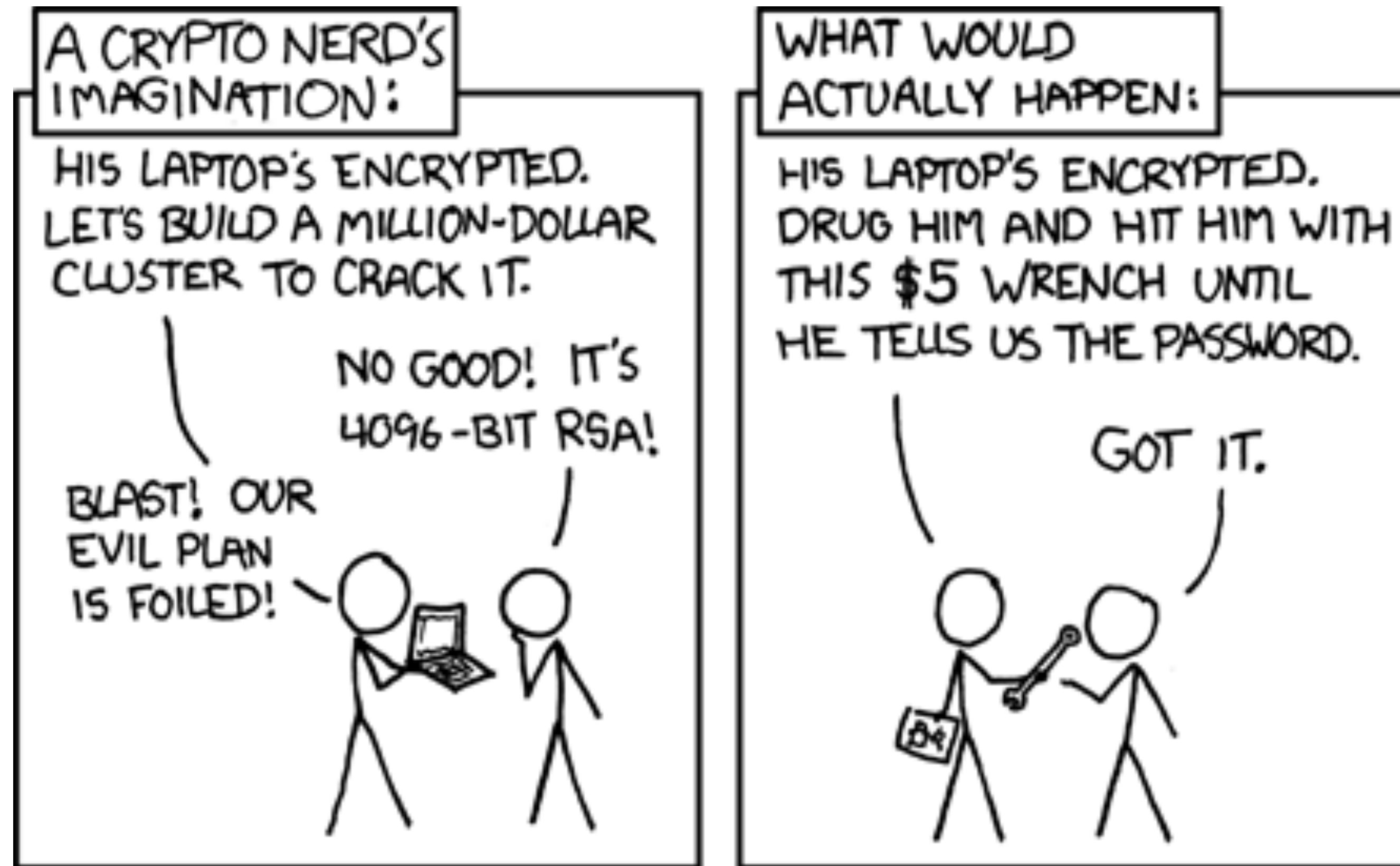
- Key exchange with good deniability
 - Alice and Bob want to generate a shared key without ever having met before
 - Assistance from a partially-trusted entity that mediates this connection
- Key evolution (aka ratcheting) with forward/backward secrecy
 - Use each key to protect just 1 message, then *delete it!*
 - Protect message privacy + integrity against device compromise in past + future
 - Generate a new key for the next message



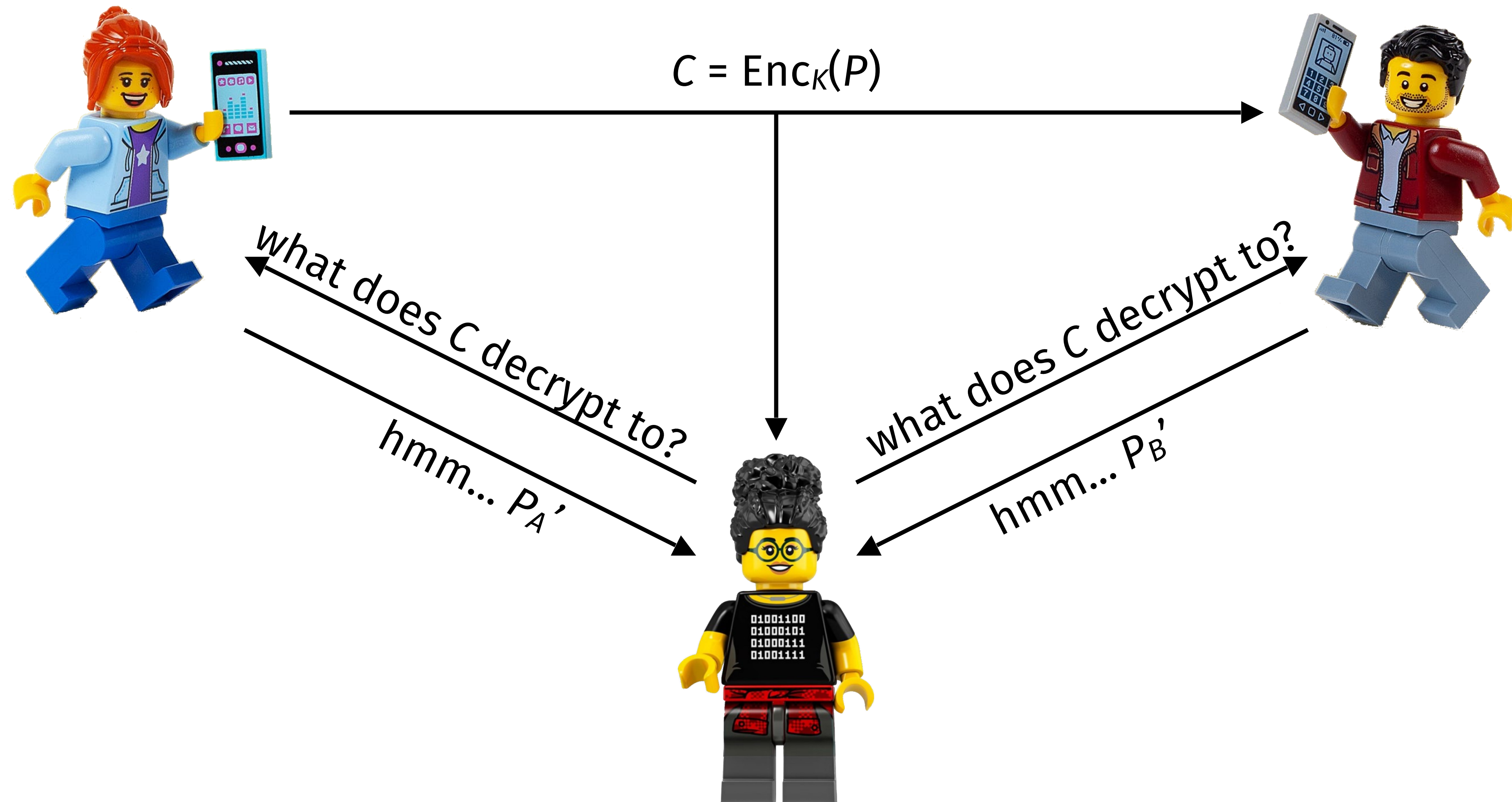
Forward and backward secrecy



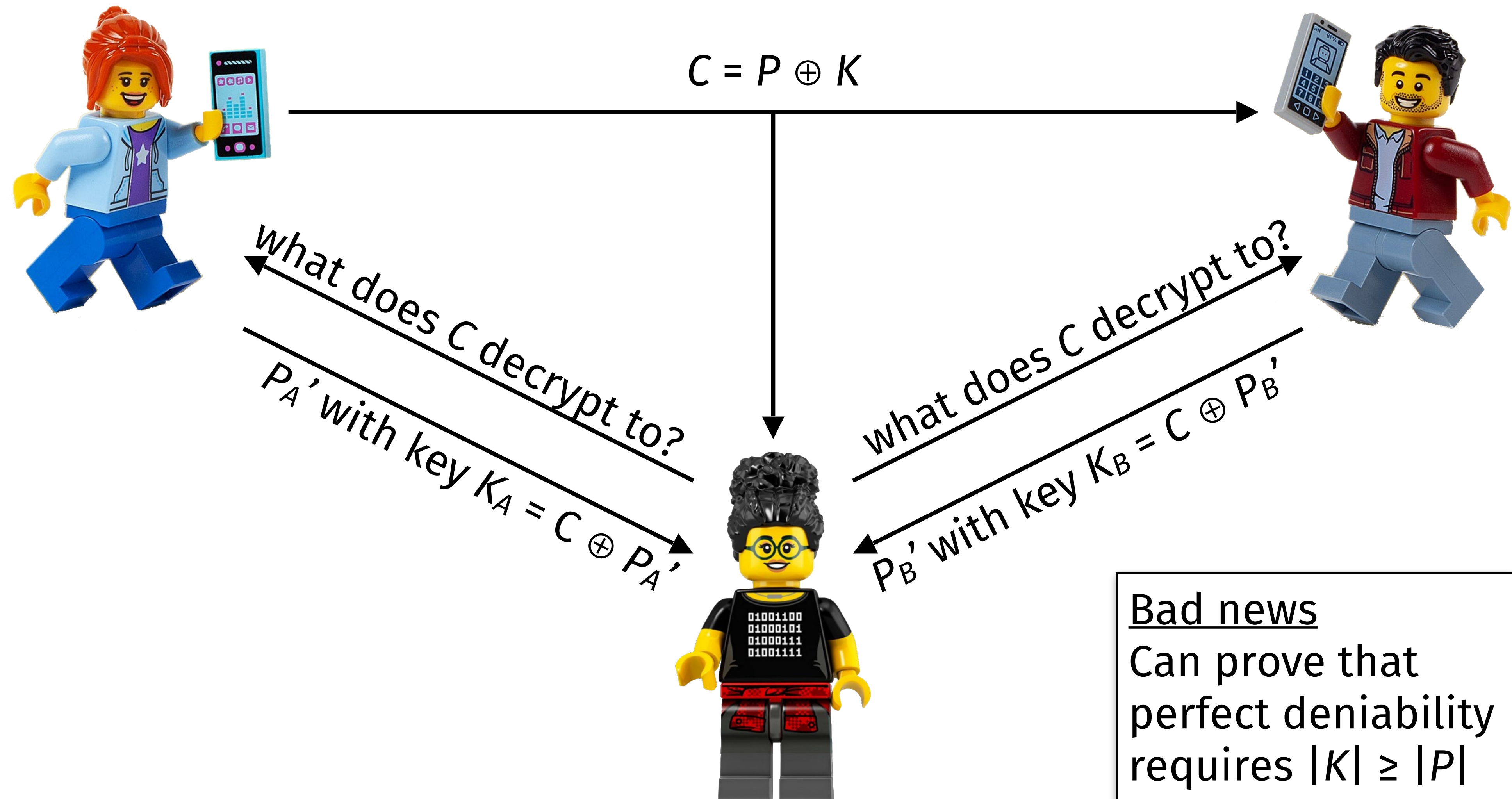
Non-deniable crypto (xkcd.com/538)



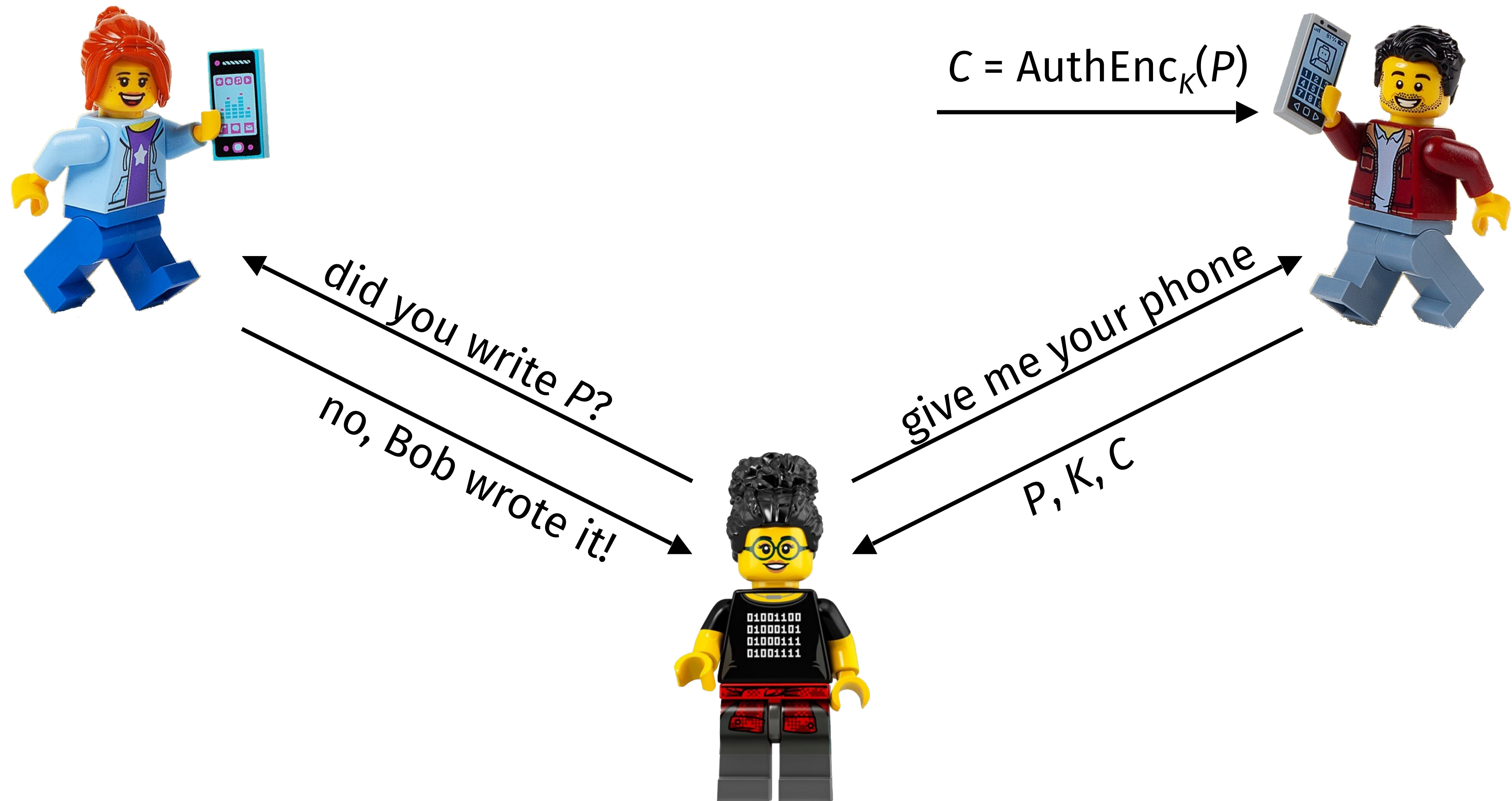
Deniable crypto = can pretend you said something else



One time pad $\rightarrow$ perfect deniability



Auth encryption → partial sender deniability



Deniability in practice

The Trump Administration Accidentally Texted Me Its War Plans

U.S. national-security leaders included me in a group chat about upcoming military strikes in Yemen. I didn't think it could be real. Then the bombs started falling.

By Jeffrey Goldberg



Andrew Harnik / Getty



[T]he DKIM authenticity stamp has been widely used by the press, primarily in the context of political email hacks. It's real, it's important, and it's meaningful.

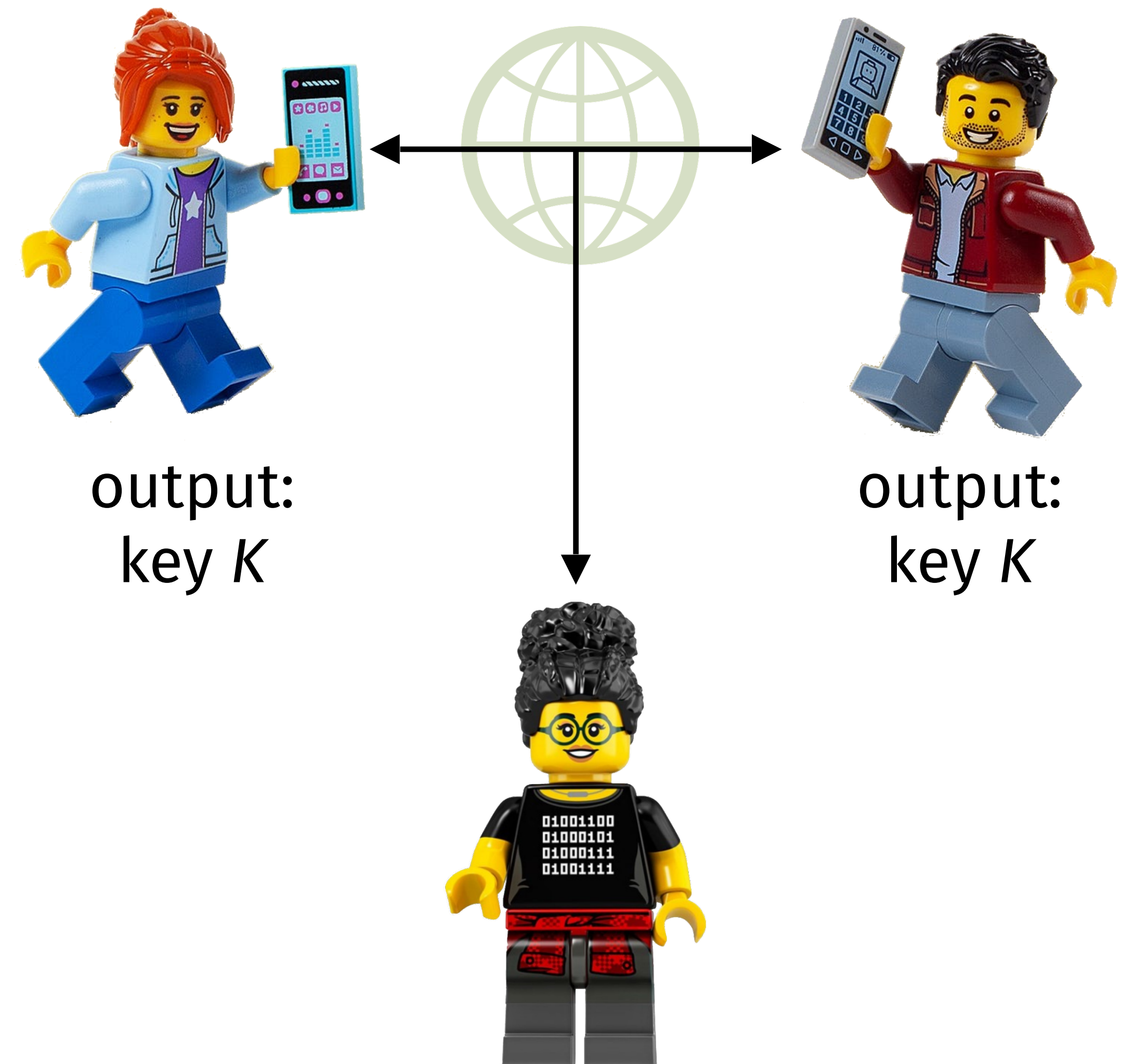
The most famous example is also one of the most divisive: back in 2016, Wikileaks published a batch of stolen emails stolen from John Podesta's Google account. Since the sourcing of these emails was murky, WikiLeaks faced a high burden in proving to readers that these messages were actually authentic. DKIM provided an elegant solution: every email presented on Wikileaks' pages publicly states the verification status of the attached DKIM signatures, something you can see today.

But the Podesta emails weren't the end of the DKIM story. In 2017, ProPublica used DKIM to verify the authenticity of emails allegedly sent to a critic by President Trump's personal lawyer Mark Kasowitz. In 2018, the Associated Press used it once again to verify leaked emails tying a Russian lawyer to Donald Trump Jr.

17.2 Authenticated Key Exchange

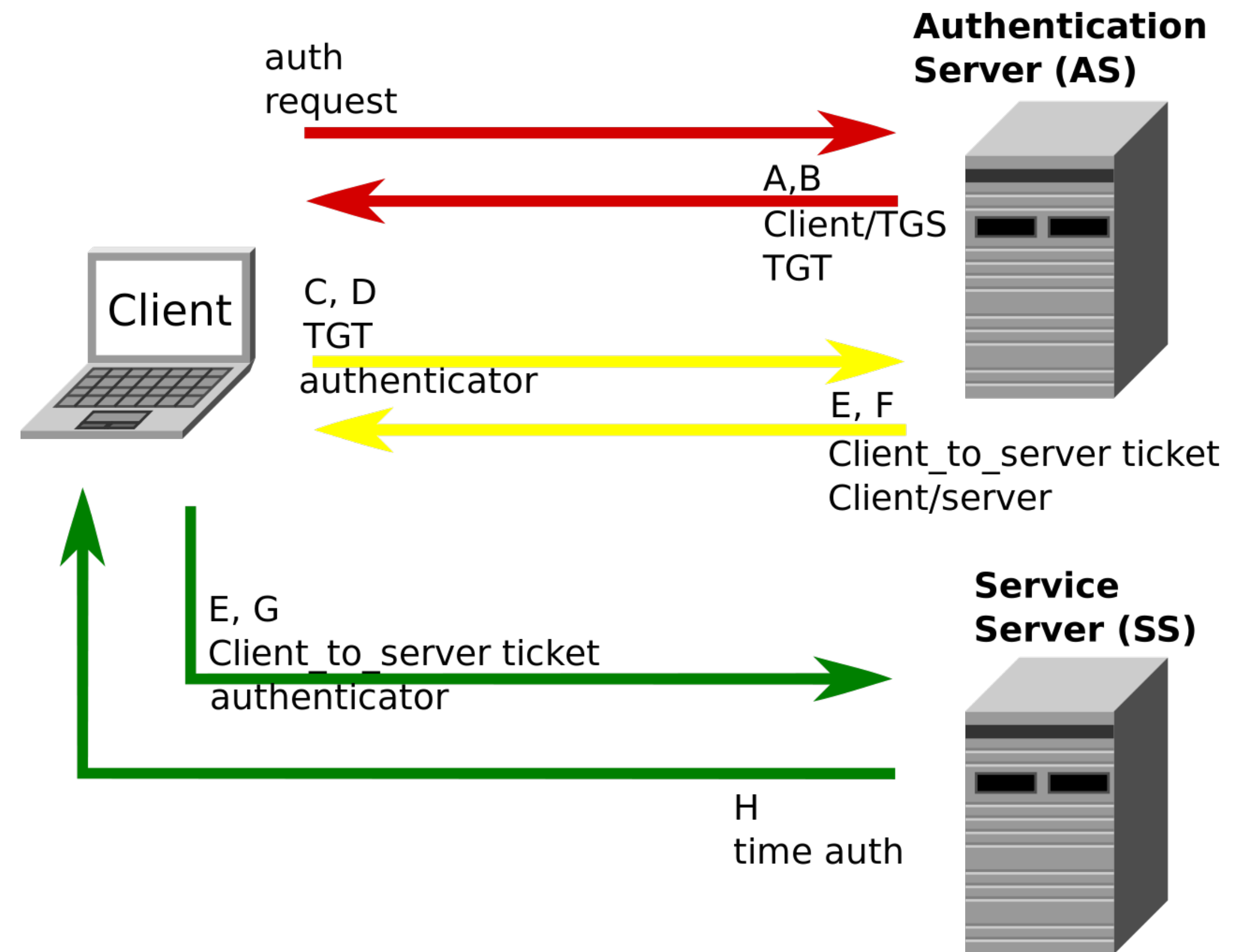
Generating the first shared secret

- Alice and Bob have
 - Never met in person, or else they could exchange a key face-to-face
 - Lack any shared secrets, or else they could run PBKDF2 on them
- They do have individual secrets!
- Question: can Alice and Bob generate a symmetric key K and keep it secret from Eve/Mallory?



One option: ask a common friend to connect you

- Suppose that
 - Alice + Bob both trust server S
 - Alice + Bob have shared symmetric keys K_{AS} , K_{BS} with the server
- Then, server can create key K_{AB} and send it to them
 - Needham-Schoeder (1978)
 - Kerberos (late 1980s)
- Mostly used within an enterprise, where auth server sees all

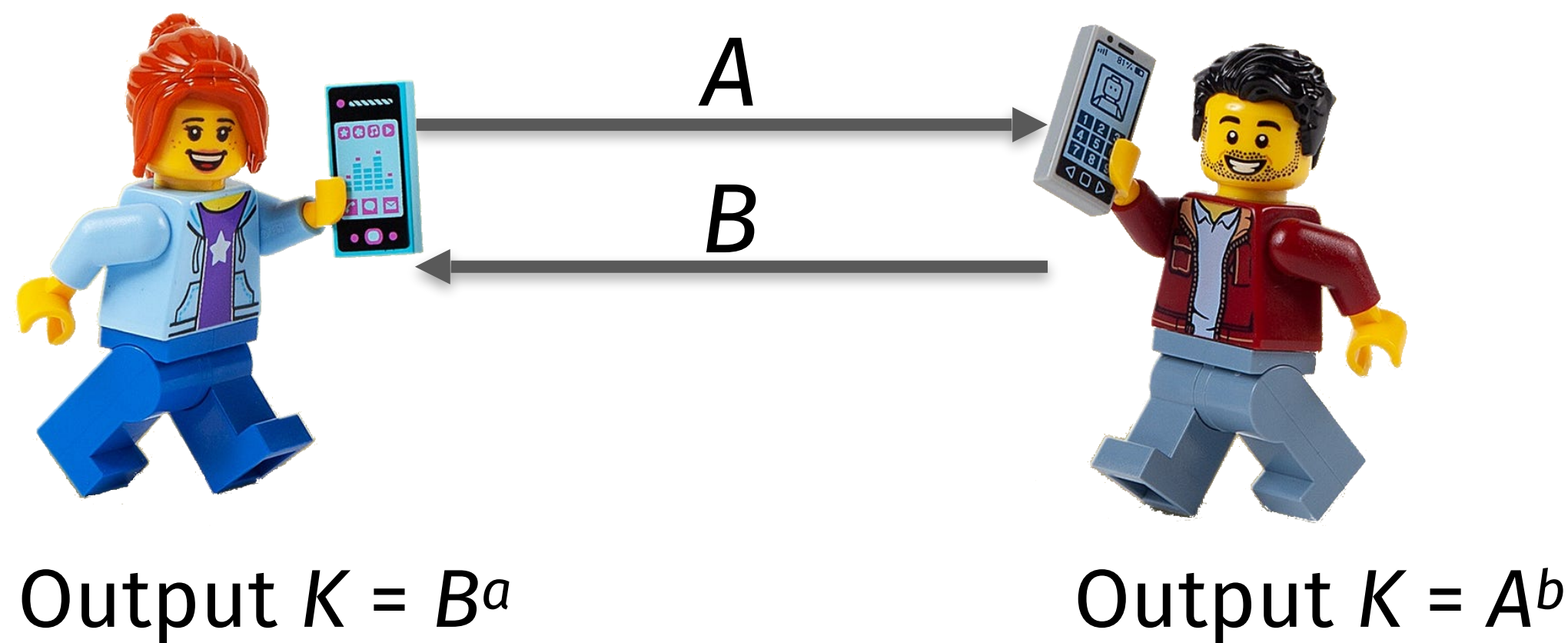


Another option: Diffie-Hellman key agreement

Protocol (against a passive Eve)

Choose a randomly
Compute $A = g^a$

Choose b randomly
Compute $B = g^b$



AuthEnc_K(P)

Delete a, K Delete b, K

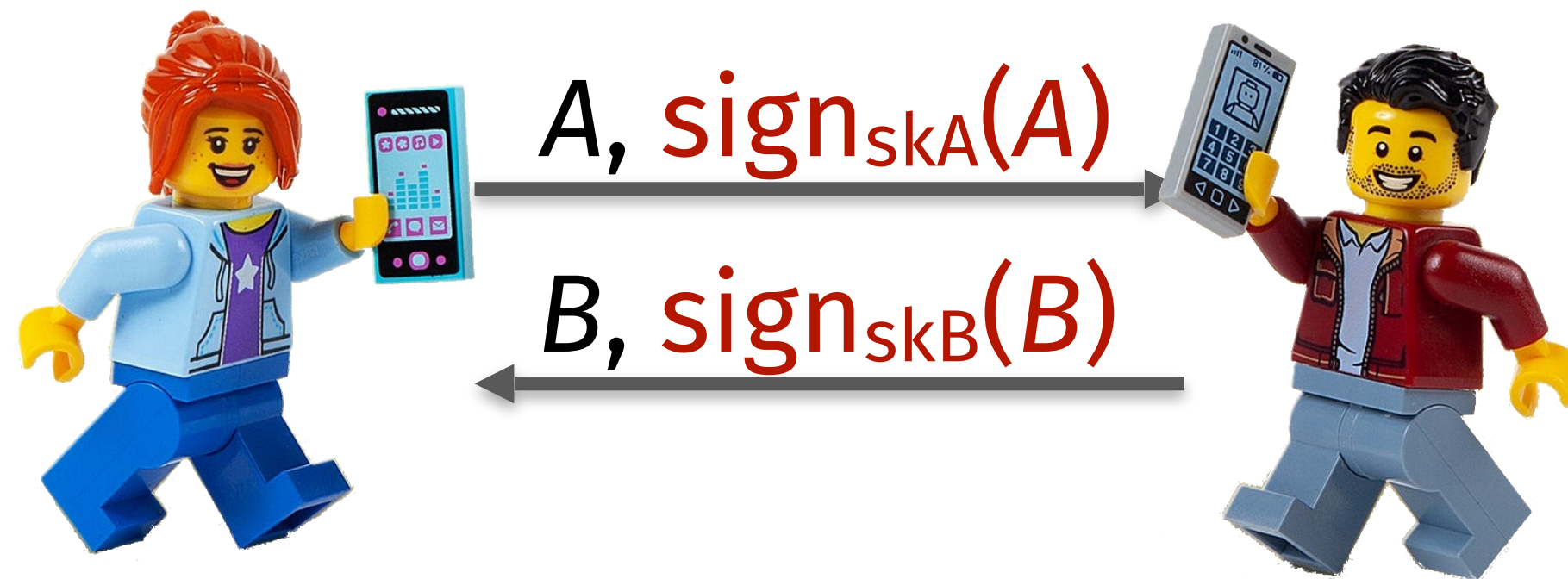
Analysis

- *Correctness*: shared secret since $A^b = (g^a)^b = g^{ab} = (g^b)^a = B^a$
- *Secrecy*: to learn K , a passive Eve given g, g^a, g^b must find g^{ab}
 - There exist mathematical spaces in which this problem is hard!
- *Forward secrecy*: Choices of a, b are ephemeral; delete afterward so even you cannot compute K

D-H + signatures = Authenticated key exchange

Choose a randomly
Compute $A = g^a$

Choose b randomly
Compute $B = g^b$

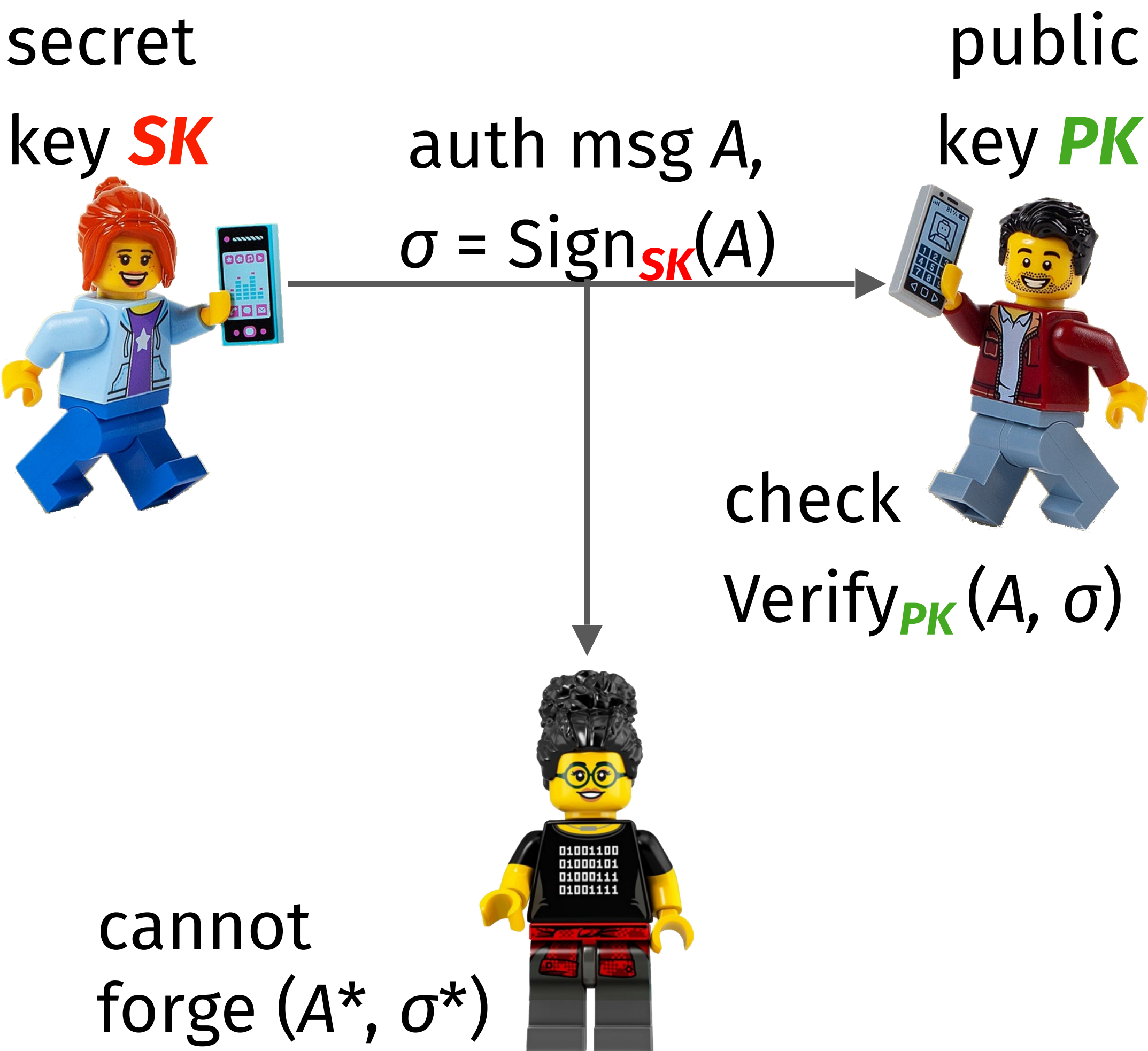


1. Alice and Bob sign their messages during Diffie-Hellman key exchange
2. Alice and Bob verify signature of each other's messages —
3. Use shared key $A^b = B^a$ for (deniable) symmetric authenticated encryption

Question: how do Alice and Bob learn each other's public keys?

17.3 Digital Certificates & the Public Key Infrastructure (PKI)

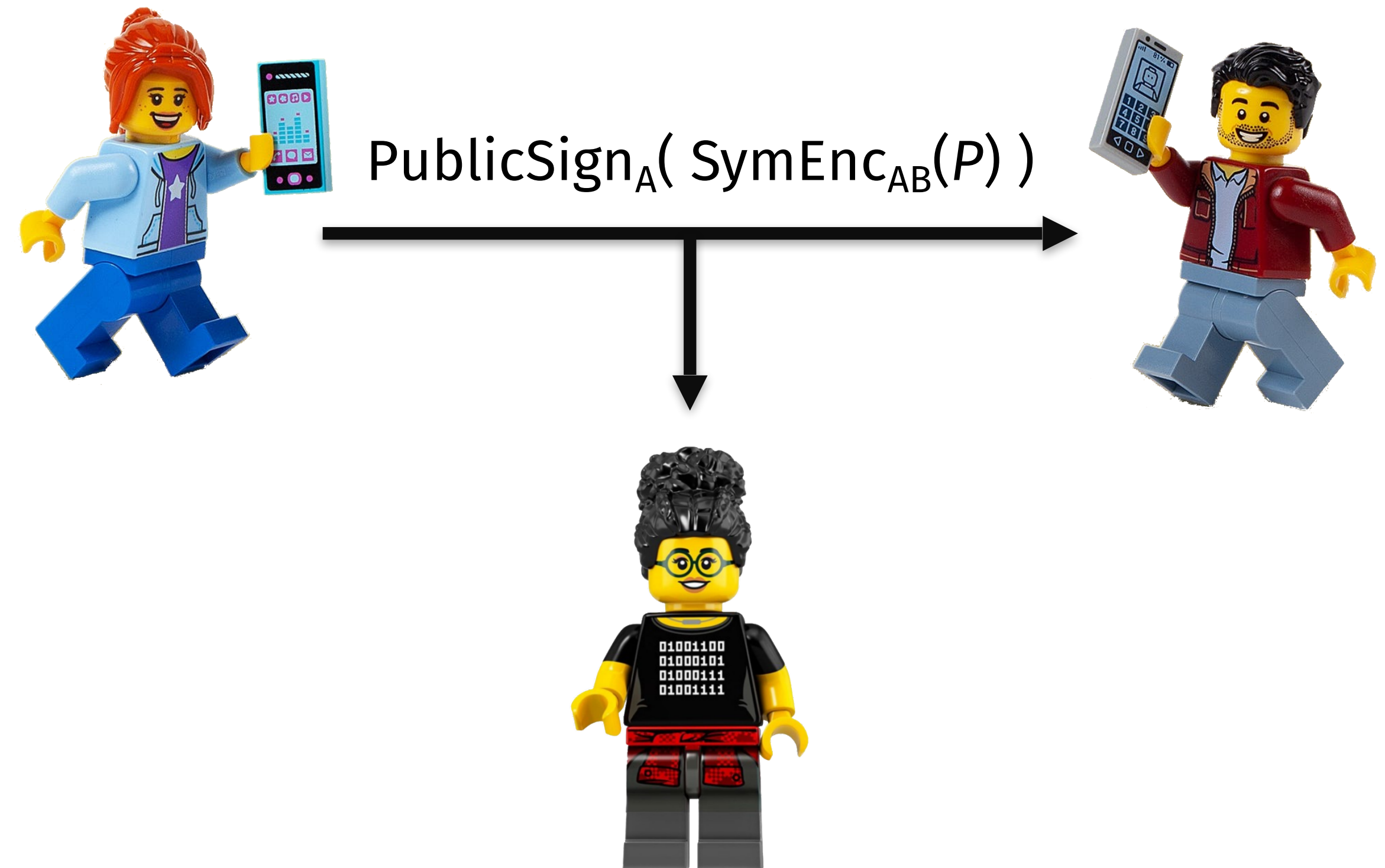
Recap: Digital signatures provide *public* authentication



<u>Property</u>	<u>MAC</u>	<u>Sign</u>
<i>Sender auth</i> : Bob knows Alice sent A	✓	✓
<i>Msg auth</i> : Bob can detect tampering	✓	✓
<i>Receiver auth</i> : Bob knows A for him	✓	✗
<i>Partial deniability</i> : Alice can deny A	✓	✗

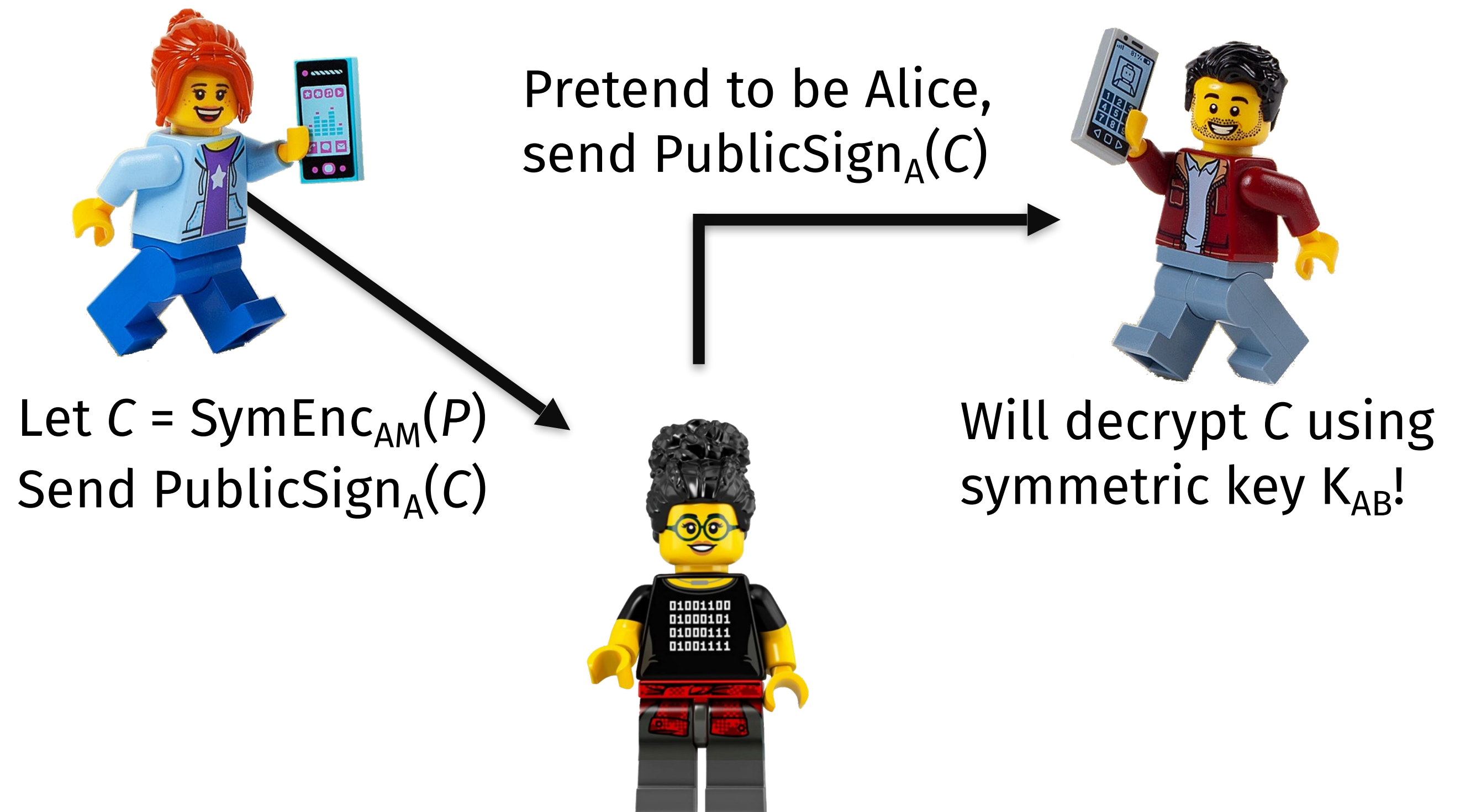
Combining symmetric encryption + public signatures

- In the symmetric case, we learned that Enc-then-MAC is best option
 - Intuition: Never expose the decryption key to an invalid message
- Does this technique work as well with public key signatures?



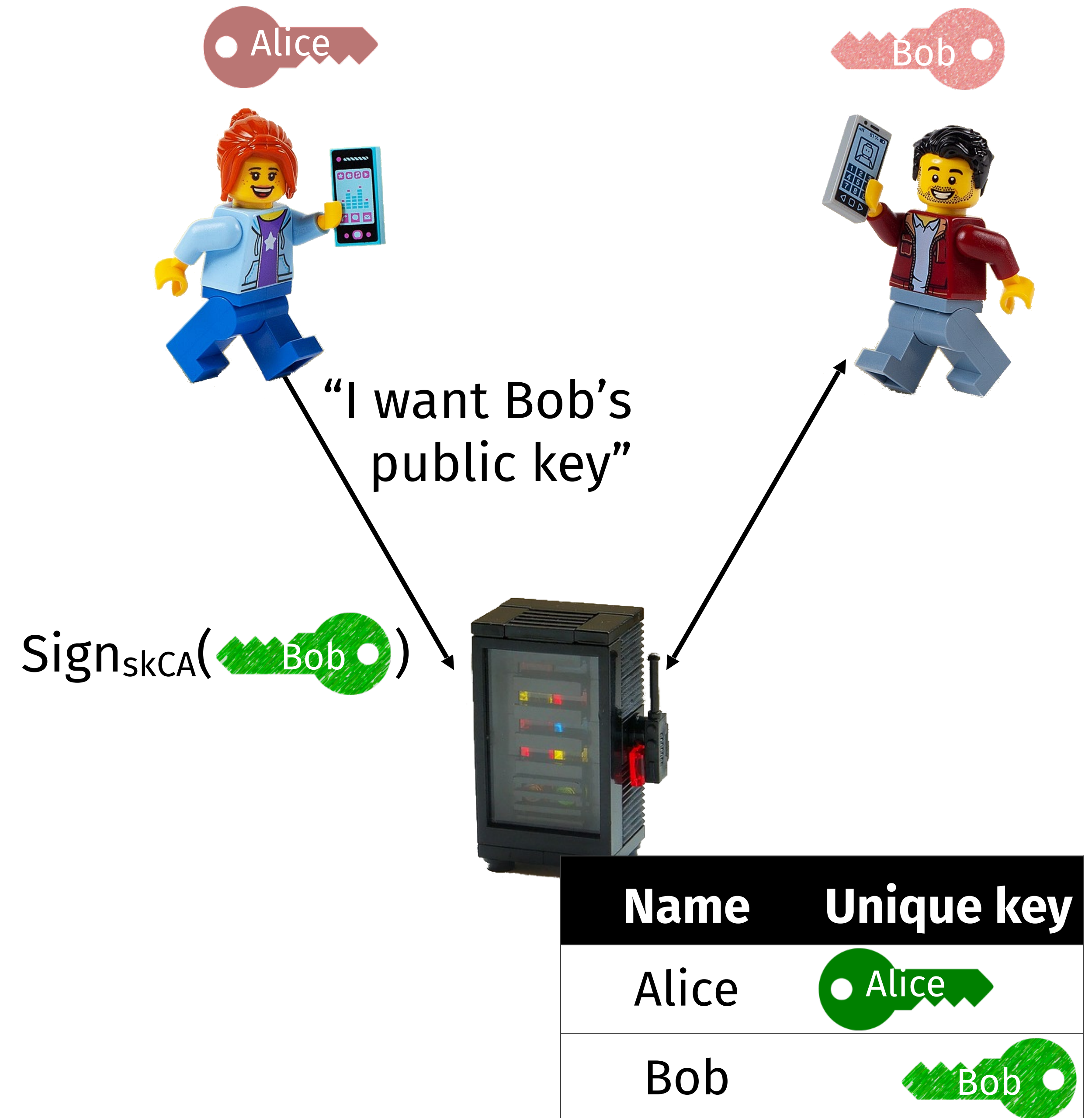
Combining symmetric encryption + public signatures

- In the symmetric case, we learned that Enc-then-MAC is best option
 - Intuition: Never expose the decryption key to an invalid message
- Does this technique work as well with public key signatures?
- Answer: No!
 - Issue: Mallory can receive ciphertexts from Alice, claim them as her own!
- Breaks crypto doom principle and led to an attack on Apple's iMessage



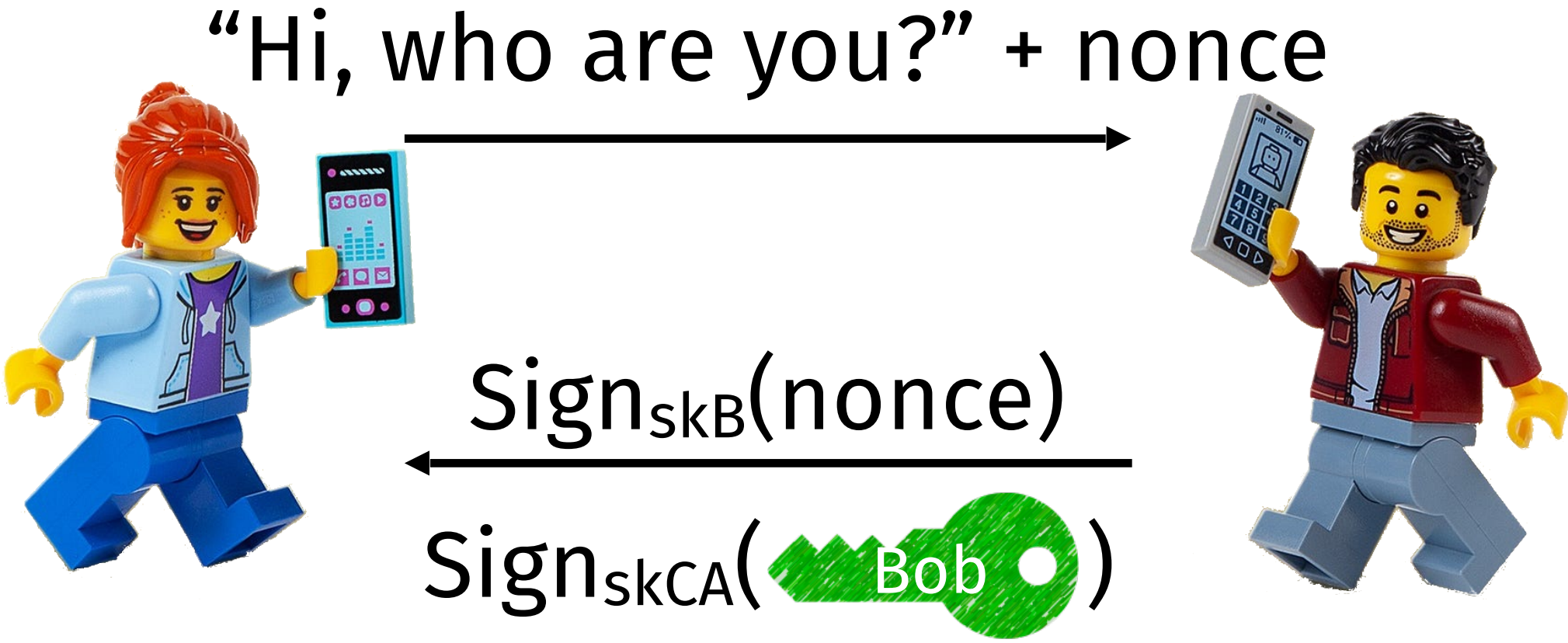
Public key infrastructure

- A *certificate authority* stores all public keys (like a phone book)
 - Server does *not* learn private keys
- Anyone can query the authority to learn someone else's key
- CA signs response *certificates* so they can be verified as legit
- Alice knows the CA's public key because it is included in her OS



PKI improved

- Alice talks with Bob, not CA
- Bob includes a certificate that the signing key belongs to him
- This figure shows a simplified version of the Transport Layer Security (TLS) handshake



Google.com in Firefox:

Technical Details

Connection Encrypted (TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256, 128 bit keys, TLS 1.2)

BU login page in Firefox (2017):

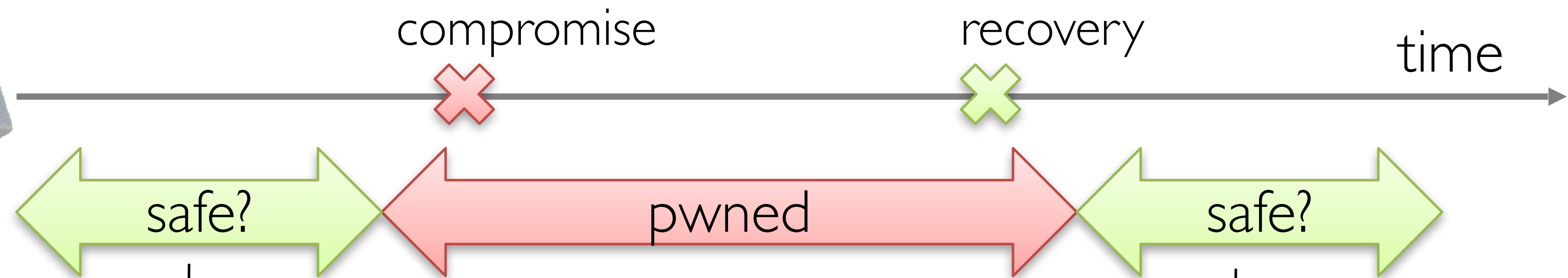
Technical Details

Connection Encrypted (TLS_RSA_WITH_AES_256_CBC_SHA, 256 bit keys, TLS 1.2)



Name	Unique key
Alice	
Bob	

What if Bob's secret signing key is compromised?

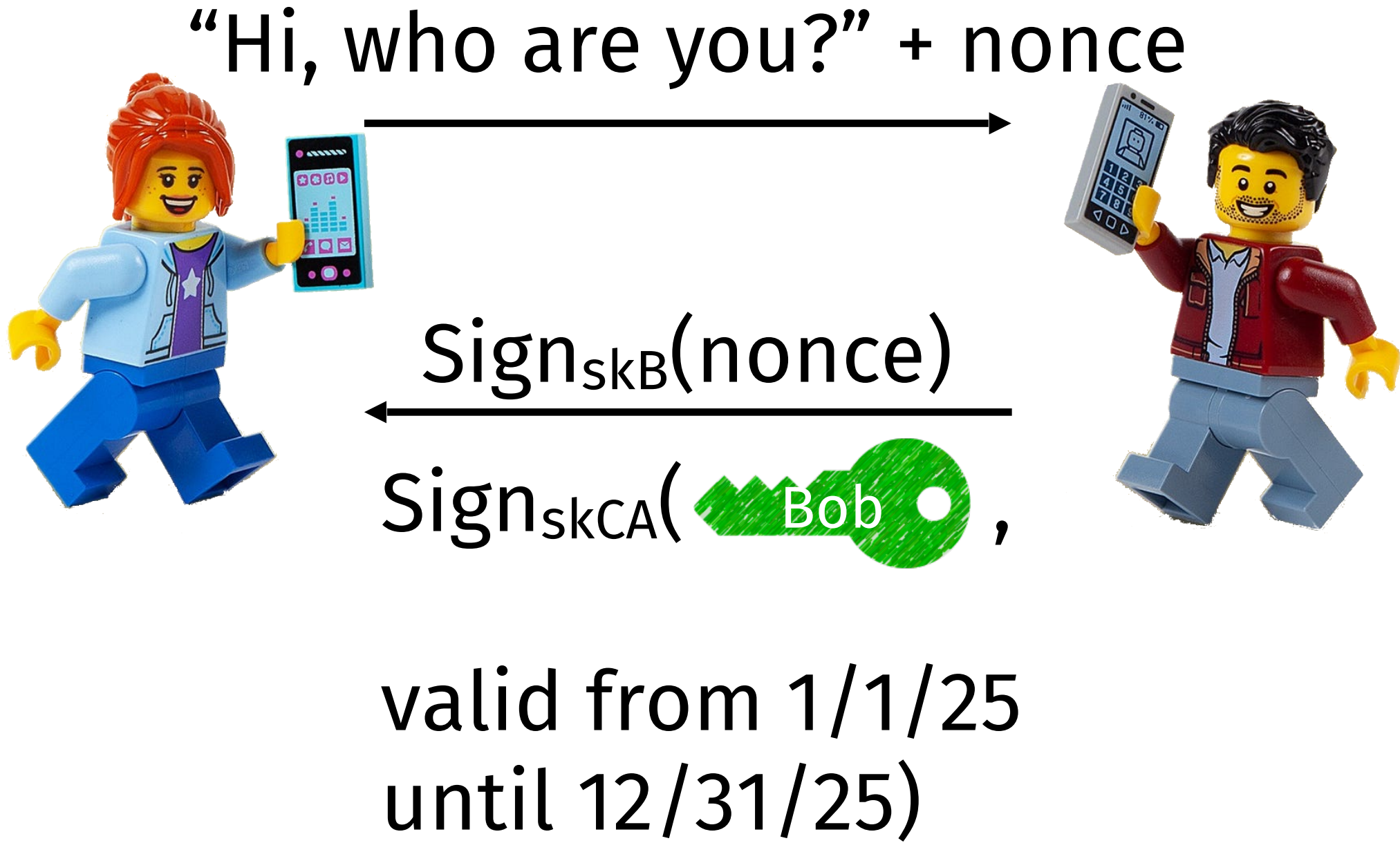


Forward (pre-compromise) secrecy
Yes! Unless Mallory has a time machine, signatures Alice verified before a breach must be valid.

Backward (post-recovery) secrecy
No. If Mallory has Bob's secret key, she can sign messages and Alice will believe they are from Bob.

Backward security technique #1: Cert expiration

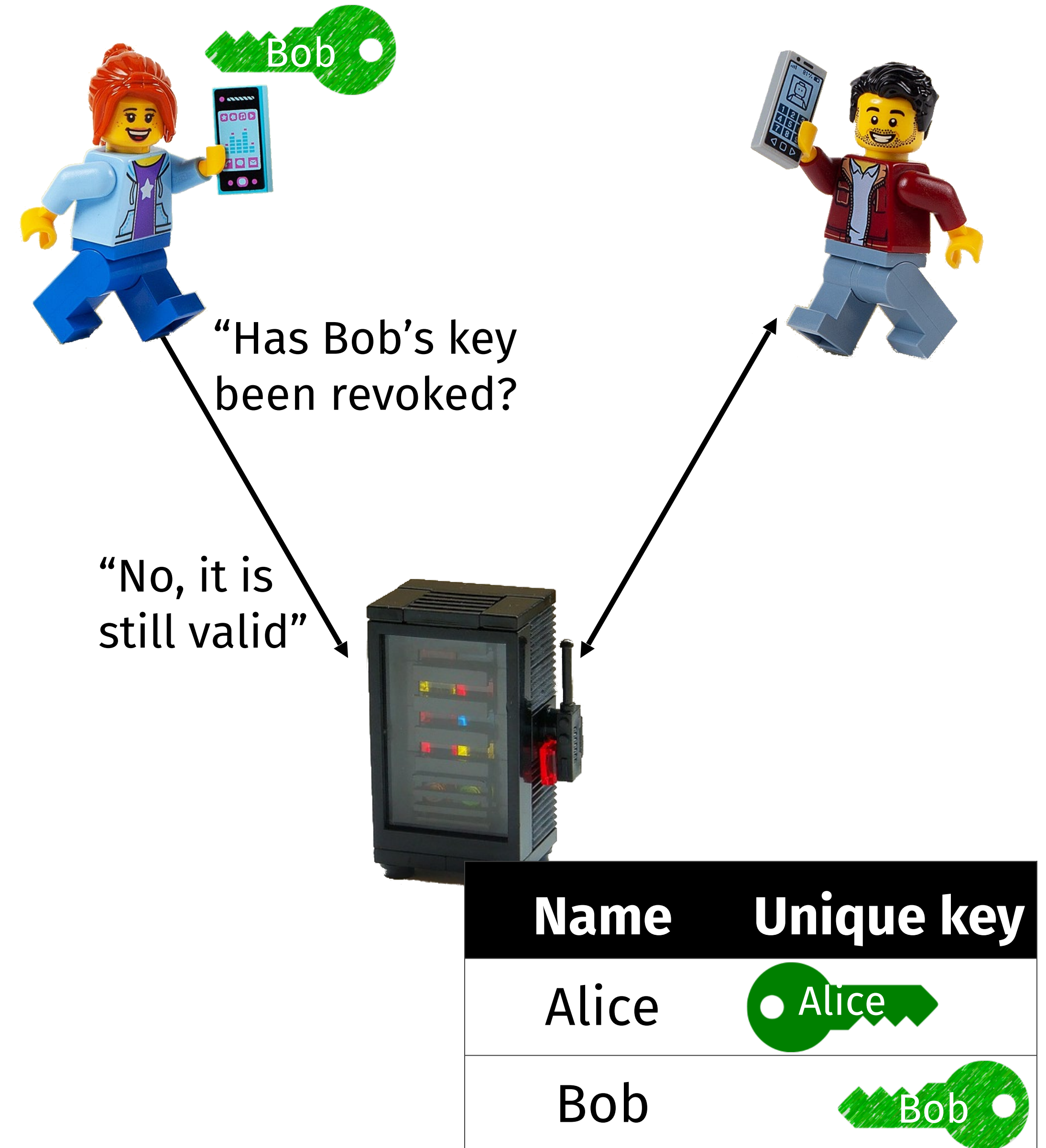
- Certificate states that Alice should only trusts Bob's key for a limited time
- Afterward, Bob must register a new public key with the CA
- (Cert expiration also helps to deal with Moore's law: keys become bigger over time)



Name	Unique key
Alice	
Bob	

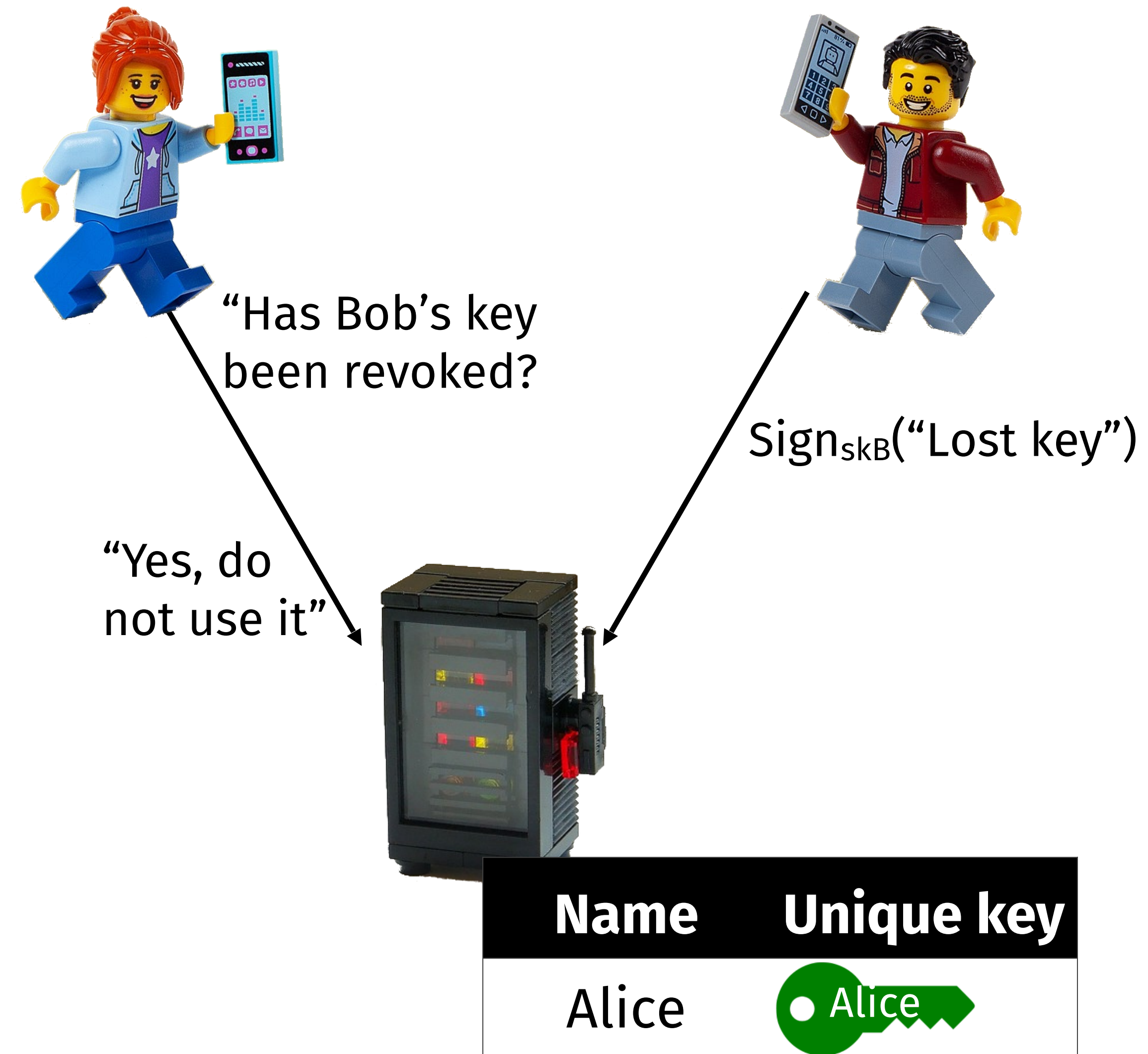
Backward security technique #2: Cert revocation

- CA binds public key to a name
- If you lose control of your public key, you should tell the CA to break this binding
- Every CA maintains a certificate revocation list that anyone can query



Backward security technique #2: Cert revocation

- CA binds public key to a name
- If you lose control of your public key, you should tell the CA to break this binding
- Every CA maintains a certificate revocation list that anyone can query



17.4 Key Evolution

Symmetric key evolution

Question: Once Alice and Bob negotiate a shared symmetric key K_{AB} for authenticated encryption, must they re-execute another (expensive) key negotiation protocol each time they want to update the key?

Basically, seek Authenticated Encryption with a key update mechanism

- KeyGen: randomly choose key K of length λ , e.g. uniform in $\{0,1\}^\lambda$
- $\text{AuthEnc}_K(\text{private } P, \text{authenticated } A, \text{nonce } N) \rightarrow \text{ciphertext } C$
- $\text{AuthDec}_K(C, A, N) \rightarrow P$ or “error”
- $\text{KeyUpdate}(K) \rightarrow K'$ where Alice + Bob agree to use K' from now onward, and cannot compute K from K'

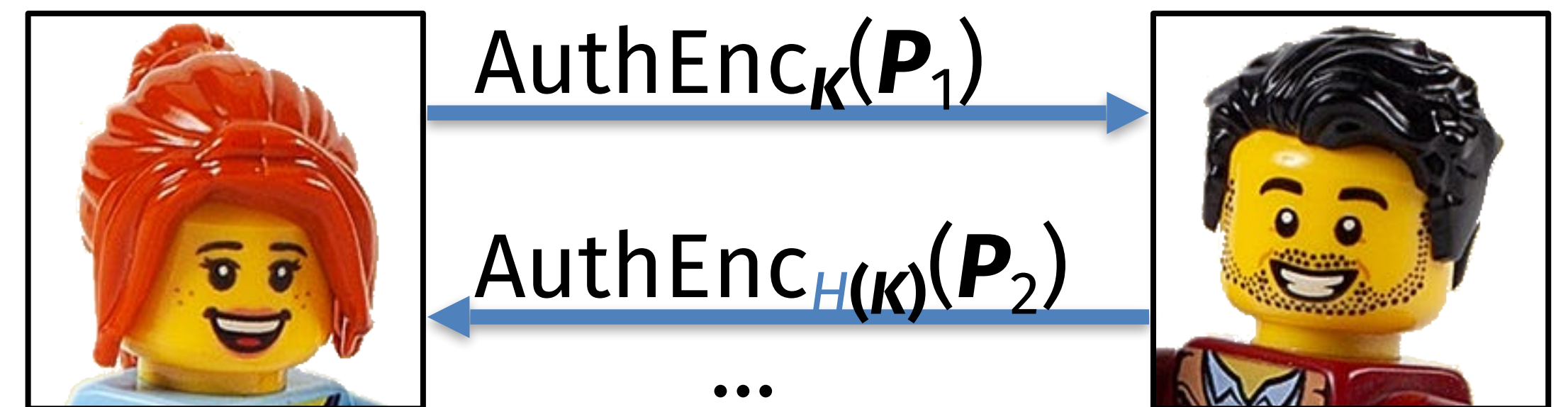
Symmetric key evolution via hash functions

Idea: Once we have a single shared key K_{AB} , expand using a chain of hash functions

$$K_{AB} \rightarrow H(K_{AB}) \rightarrow H(H(K_{AB})) \rightarrow H(H(H(K_{AB}))) \rightarrow \dots$$

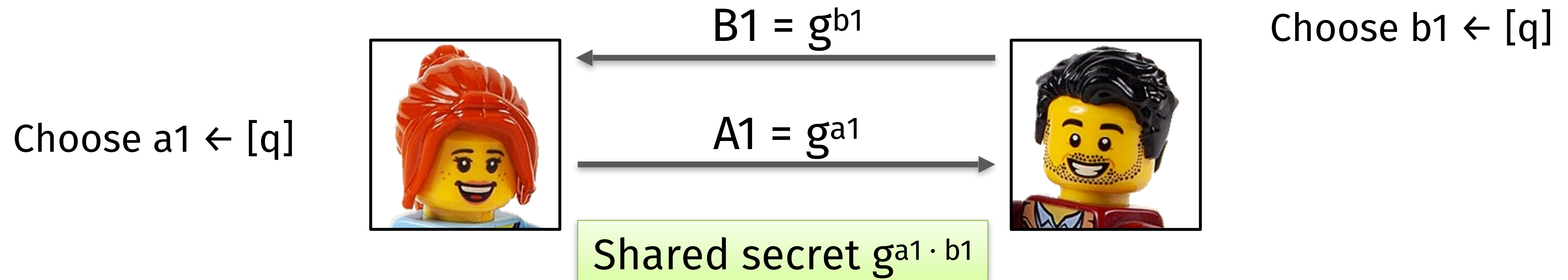
Algorithm:

- Alice + Bob agree on key K_{AB} to use for auth enc
- After some time has passed, they can evolve their key by updating $K \leftarrow H(K)$
 - Here, “time” can denote actual wall-clock time or a message counter
 - Alice + Bob must stay in sync, or else the chain breaks & they must redo key agreement
- Crucially, they ensure that old values of K are deleted from their system! Evolution relies on the fact that Mallory cannot steal something that isn't around to be stolen



Public key evolution

2 rounds of Diffie-Hellman create a shared secret, and 1 round can update it!



17.5 Signal's Double Ratchet

Double ratchet rules

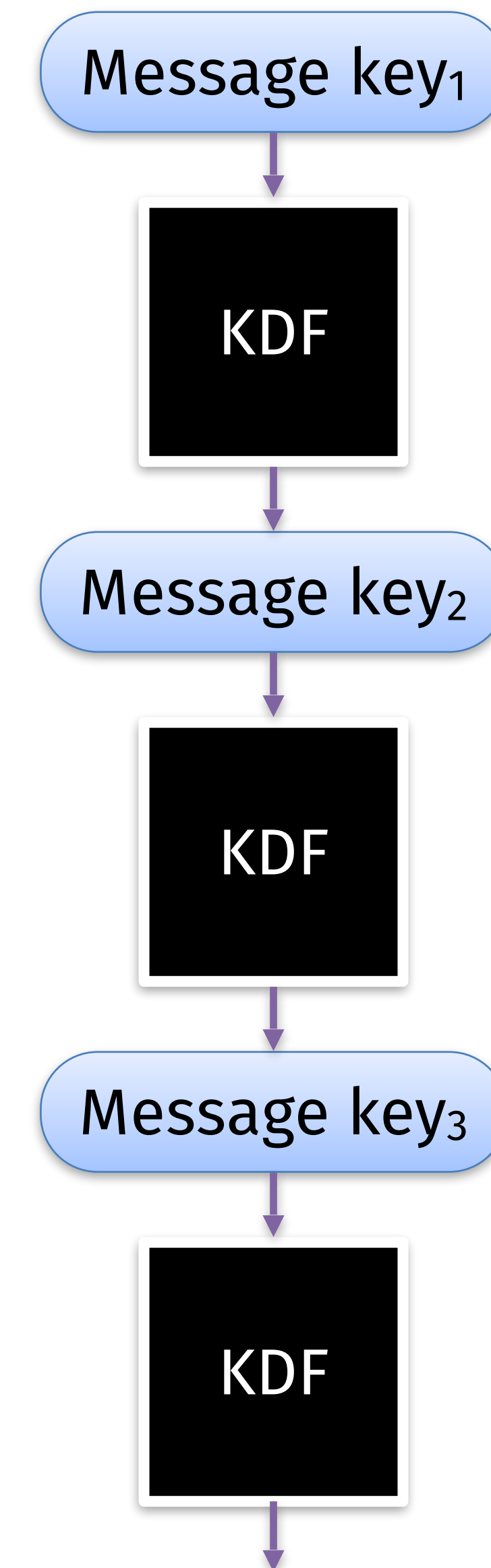
Alice and Bob maintain two sets of keys: one for Alice-to-Bob messages, and another for Bob-to-Alice messages

1. When a message is sent or received, a *symmetric ratchet* KDF step is applied to the sending or receiving chain to derive a new message key
2. When alternating the direction of communication, a *public ratchet* step updates the chain keys that are used in the symmetric ratchet

Signal messaging protocol (simplified)

1. Key evolution

- Each key encrypts 1 msg, then evolved + deleted
- Keys are forward secure but not backward secure



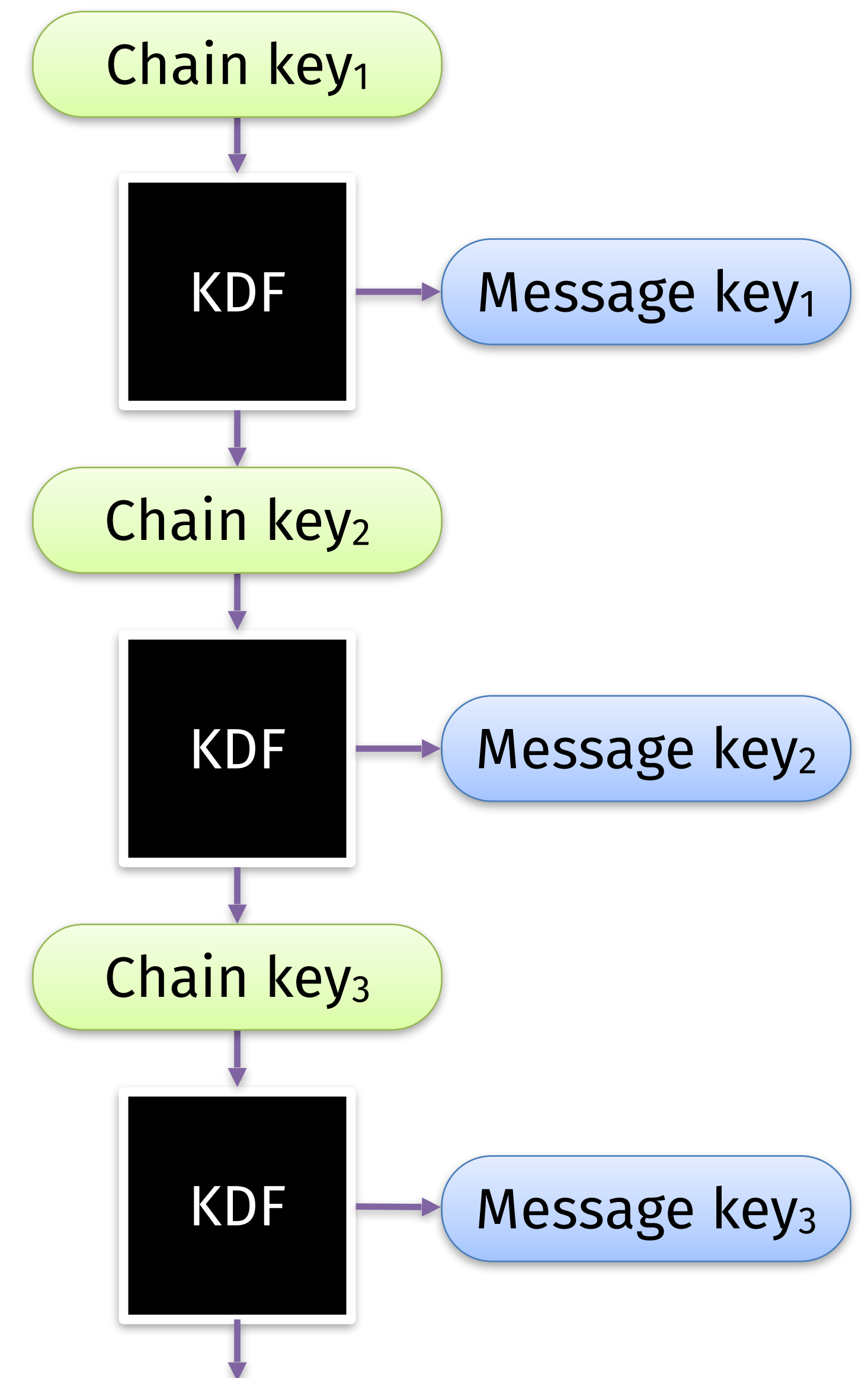
Signal messaging protocol (simplified)

1. Key evolution

- Each key encrypts 1 msg, then evolved + deleted
- Keys are forward secure but not backward secure

2. Key derivation

- Message keys now forward + backward secure
- “Message keys aren't used to derive other keys... useful for handling lost/out-of-order messages”



Signal messaging protocol (simplified)

1. Key evolution

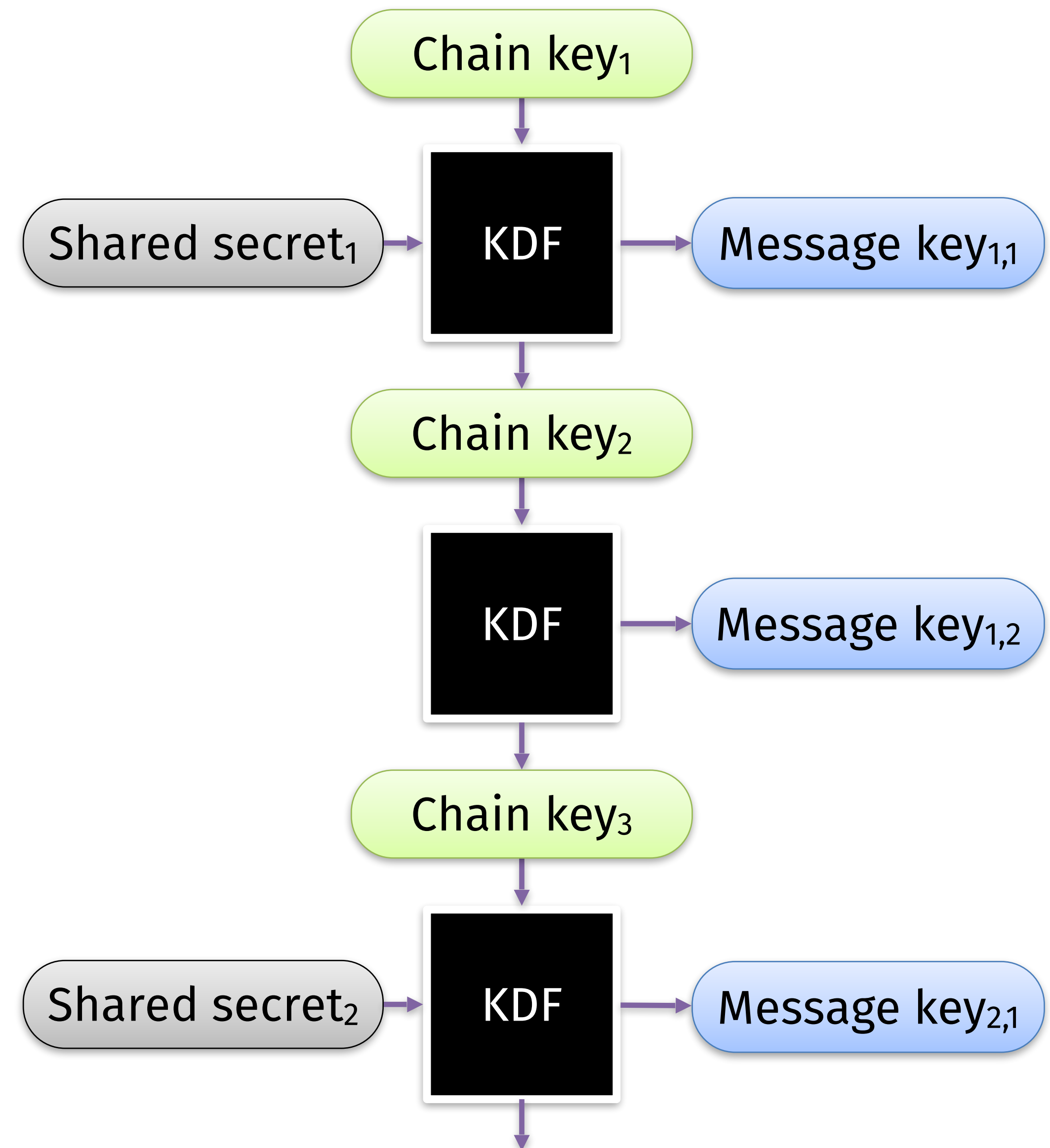
- Each key encrypts 1 msg, then evolved + deleted
- Keys are forward secure but not backward secure

2. Key derivation

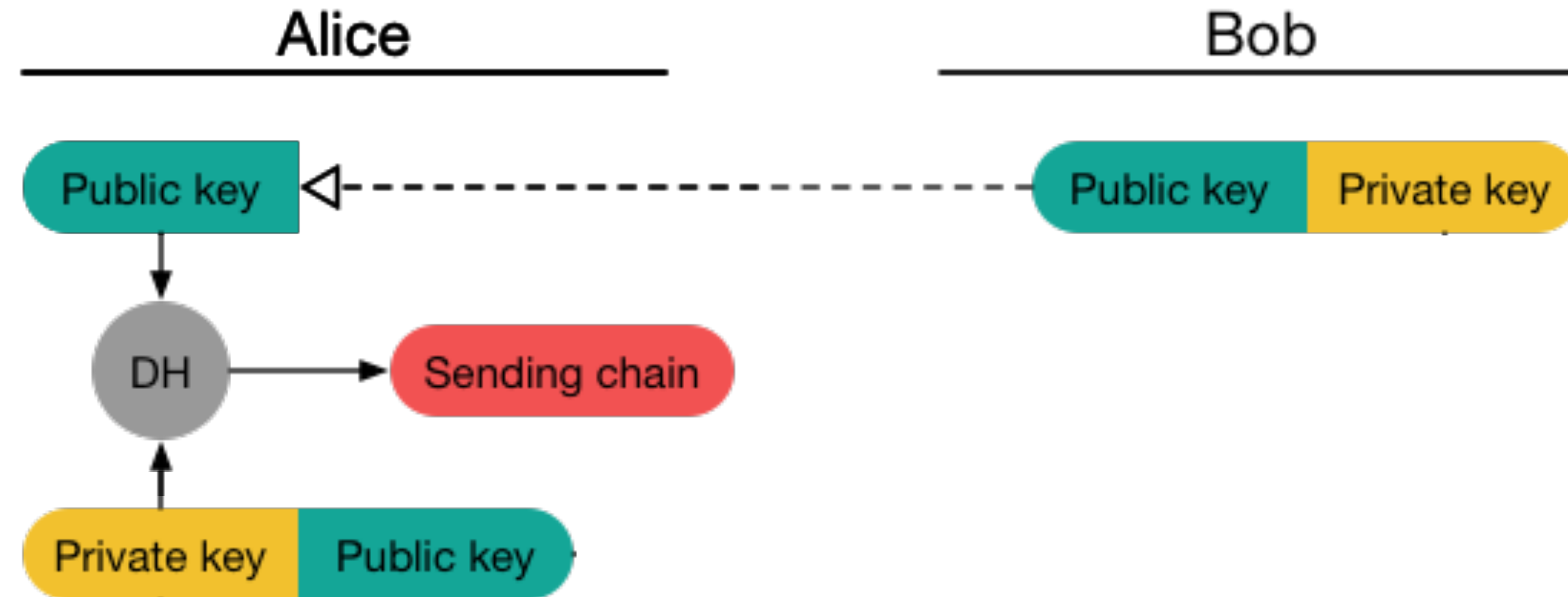
- Message keys now forward + backward secure
- “Message keys aren't used to derive other keys... useful for handling lost/out-of-order messages”

3. Key ratcheting

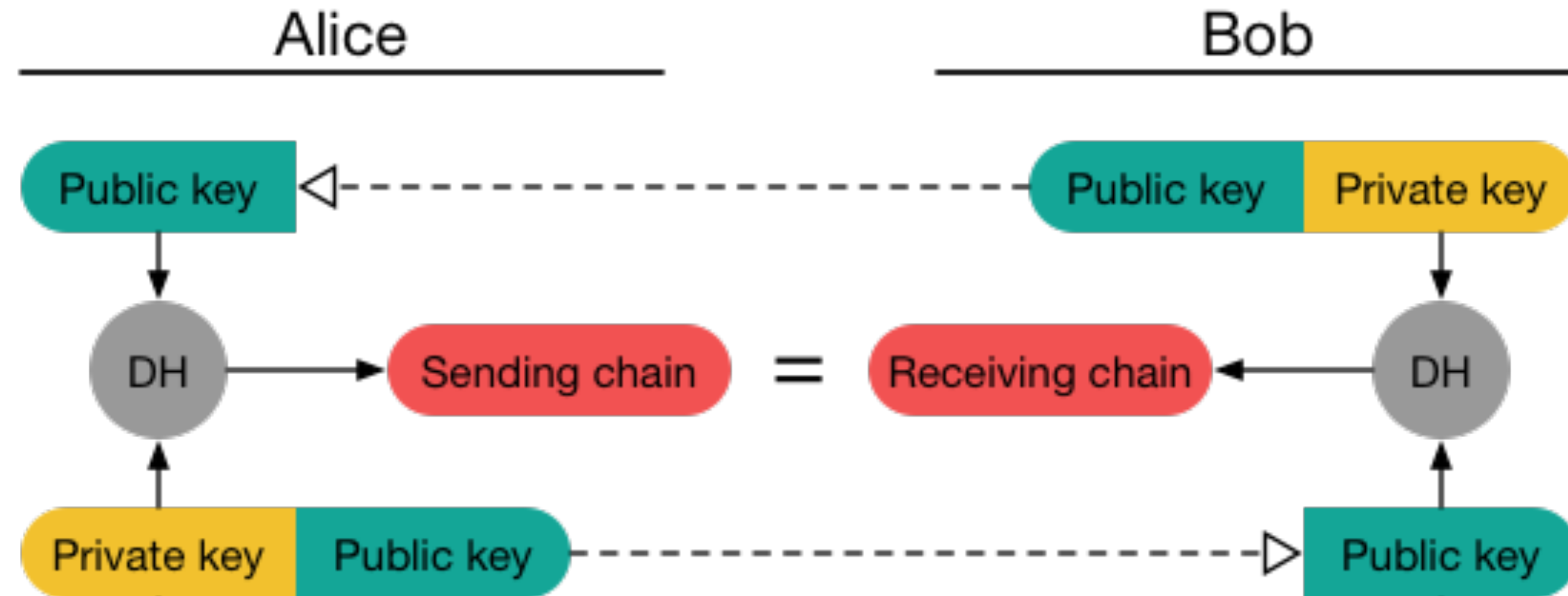
- Periodically build new D-H shared secrets
- If adv doesn't know shared secret, then recover from losing chain key (backward secrecy)



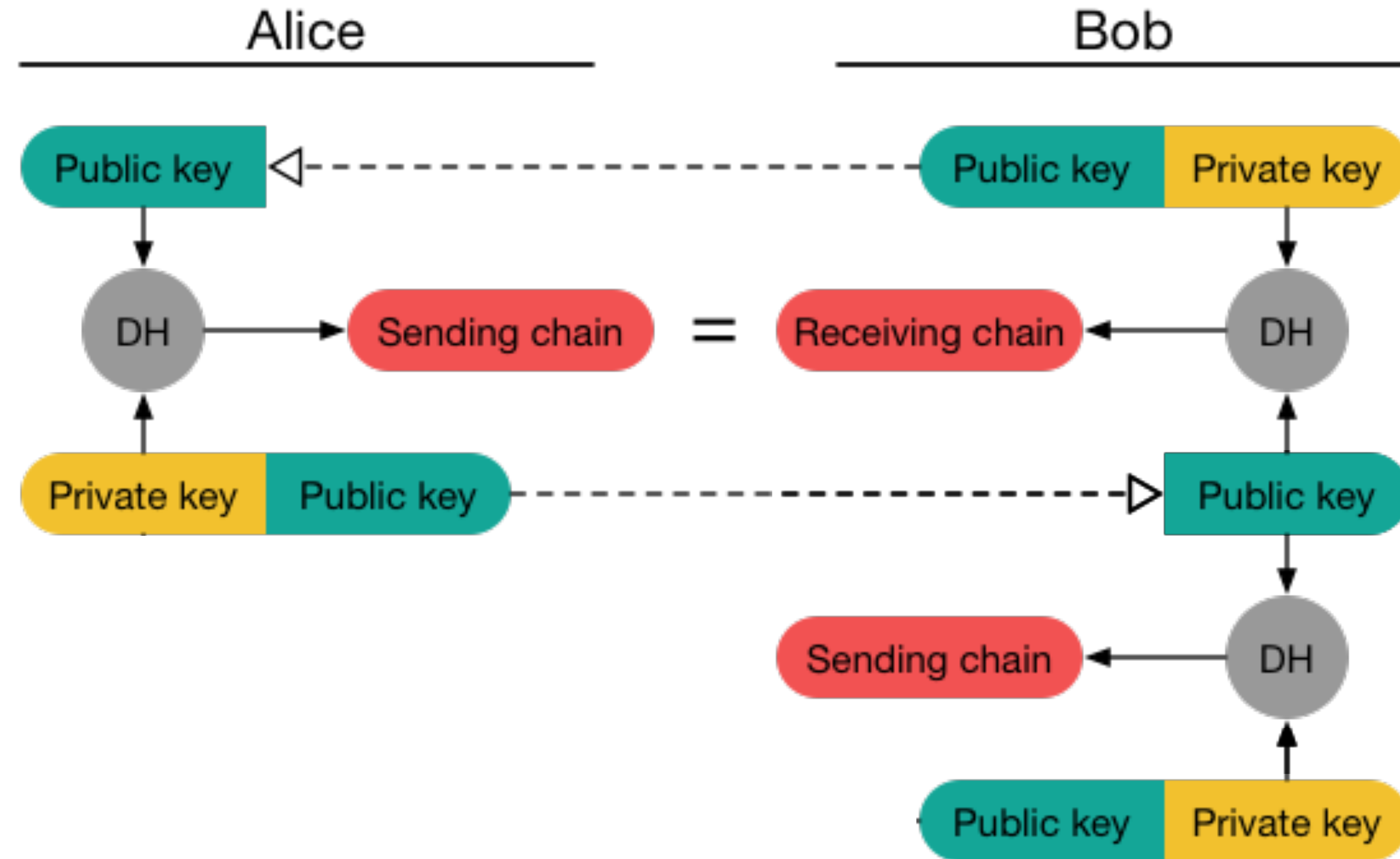
Public ratchet seeds symmetric ratchets (one per direction)



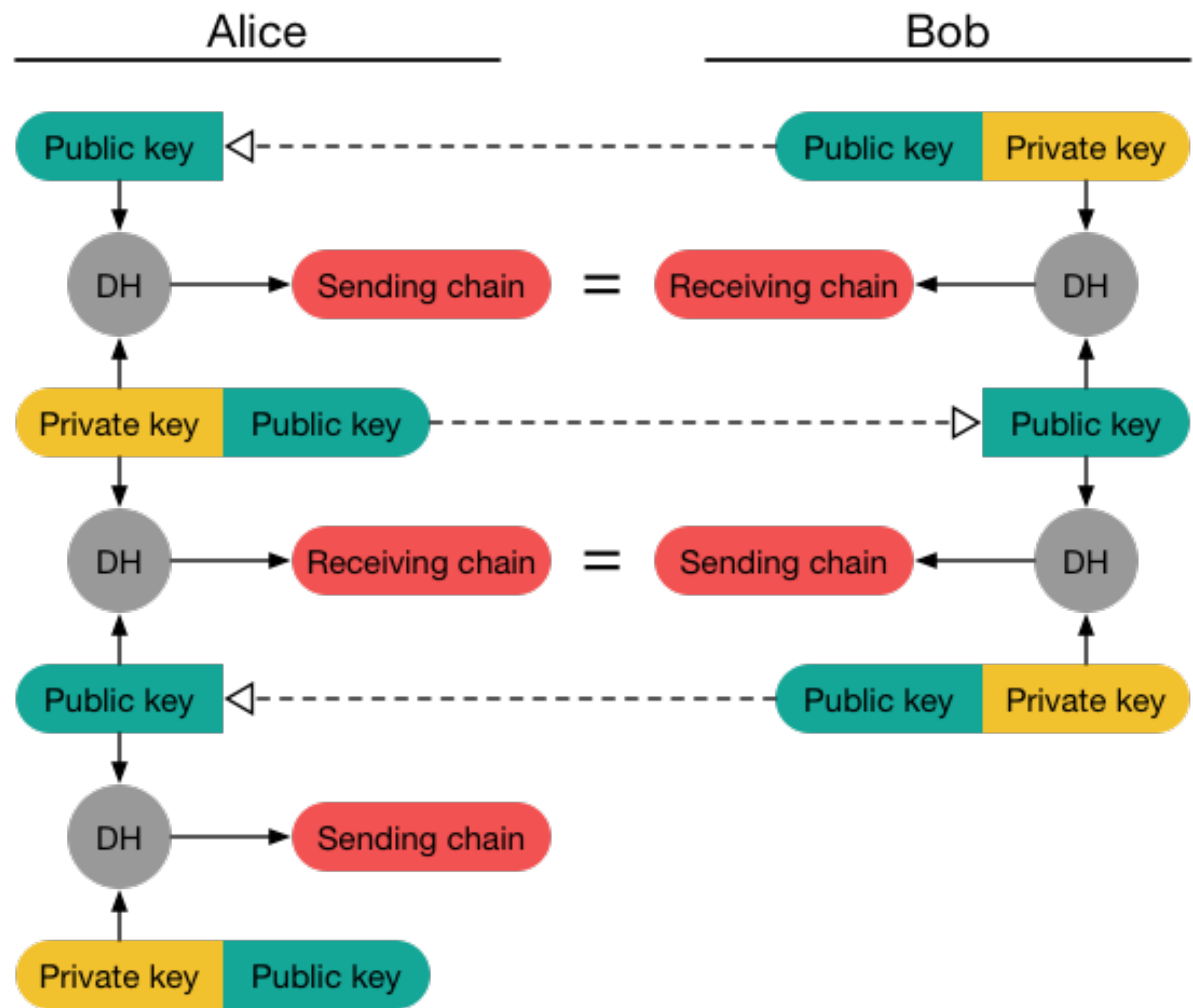
Public ratchet seeds symmetric ratchets (one per direction)



Public ratchet seeds symmetric ratchets (one per direction)

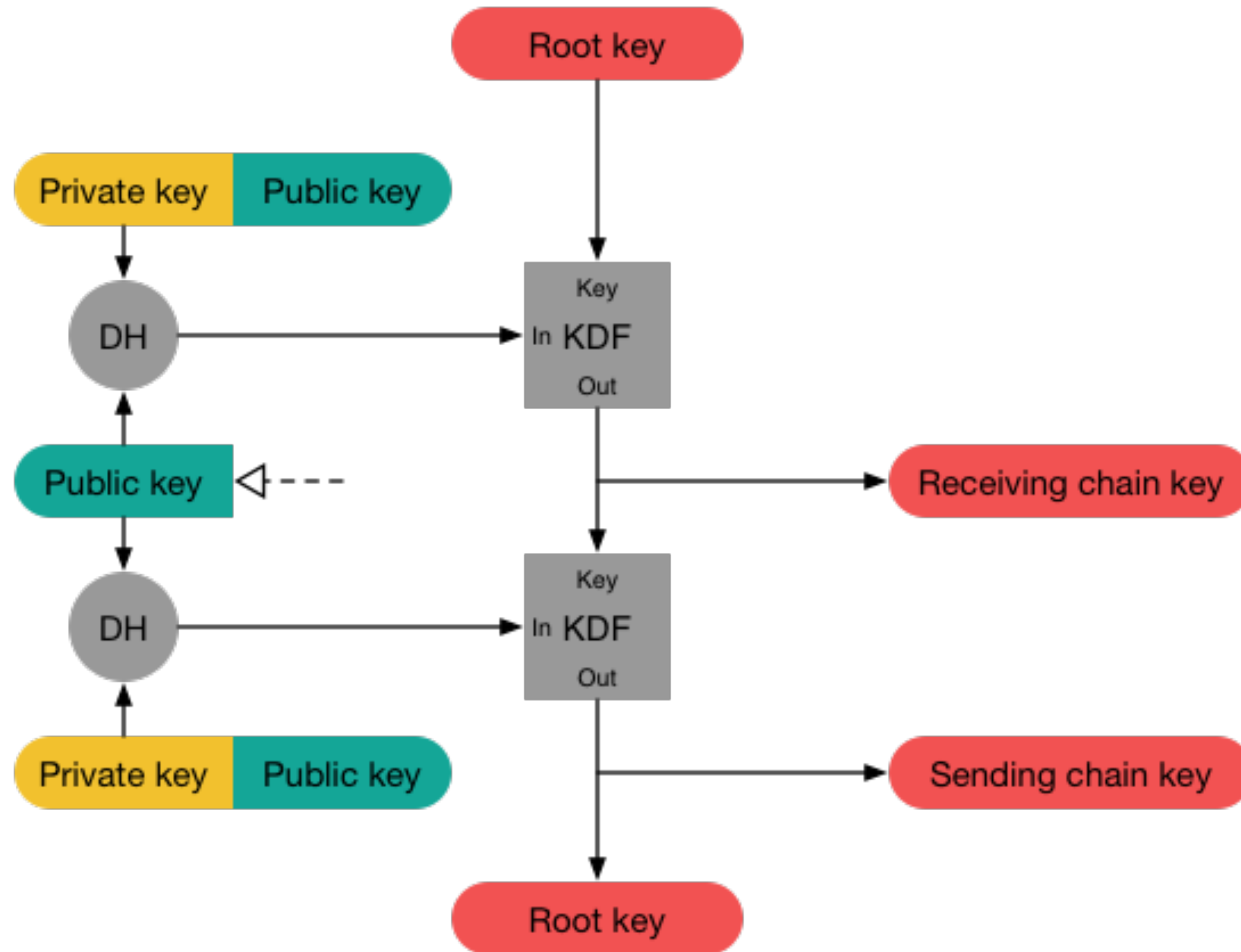


Public ratchet seeds symmetric ratchets (one per direction)

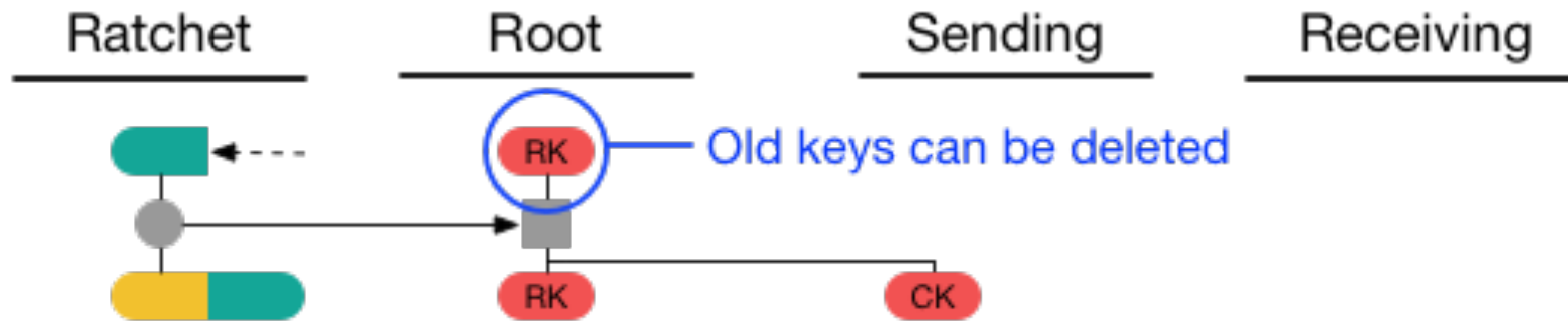


17.6 Adding Root Keys

Add a third chain to improve post-compromise secrecy



Alice's full state



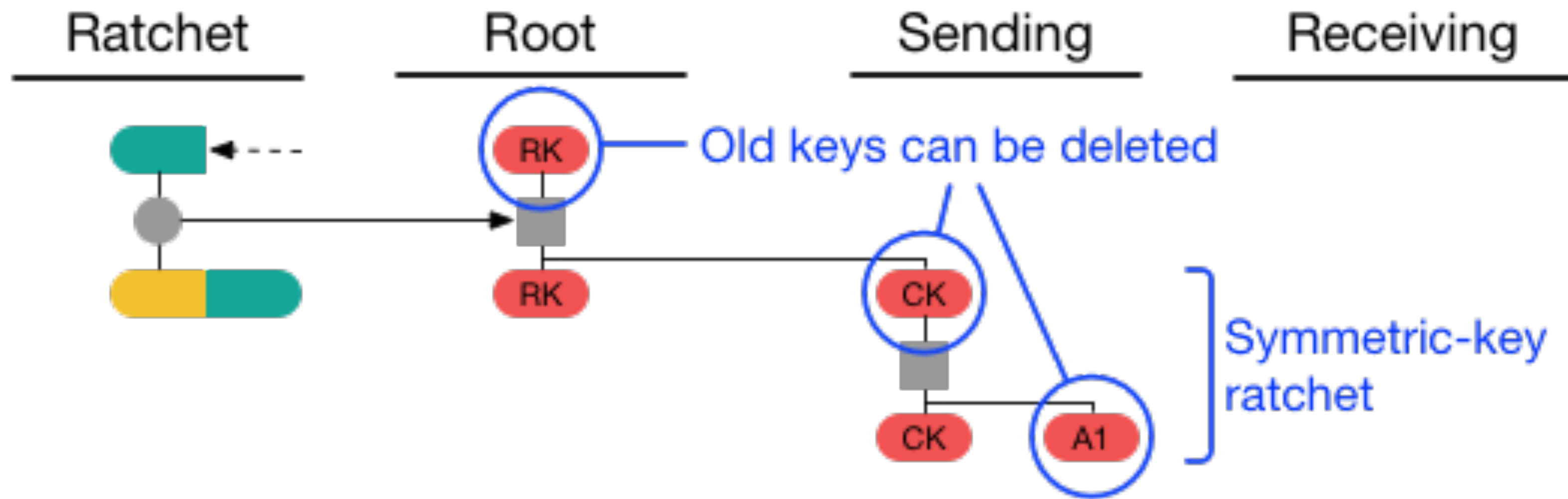
Alice has been initialized with

1. Bob's ratchet public key
2. A shared secret which is the initial root key

From this info, Alice derives

1. A new root key, whose “constant” *depends* on the public ratchet
2. The first sending key chain

Alice's full state



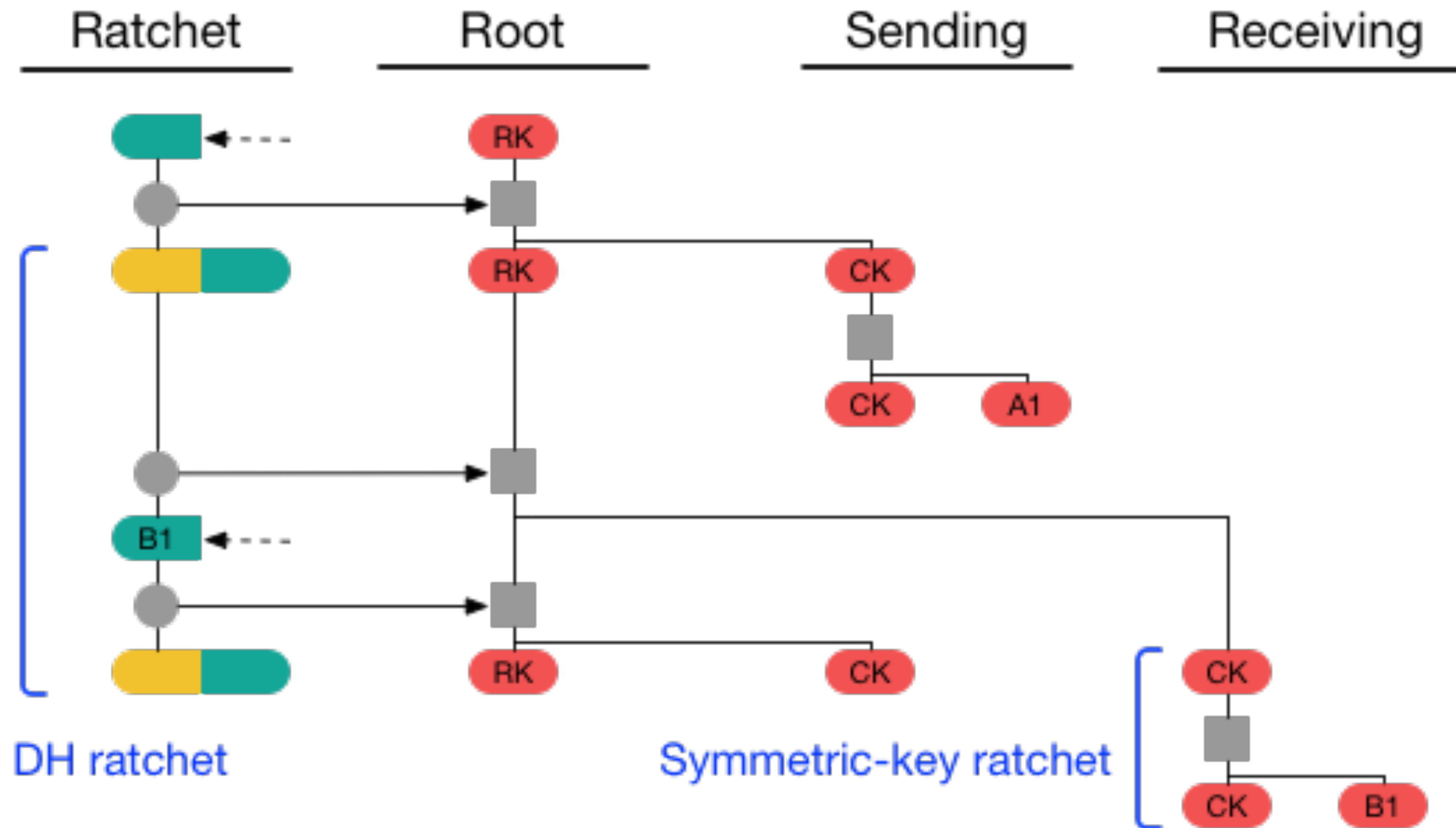
Alice sends her first message by

1. Ratcheting the sending key
2. Using the resulting *single use* message key to protect her new message

Engineering question:

Note that Bob must send  before Alice can send Bob a message.
So, how can Alice send messages if Bob isn't online?

Alice's full state



Suppose Bob sends a message next

1. Alice now has the other half of the key pair necessary to start a receiving chain
2. Bob's message includes a new , so Alice can perform a public ratchet to make a new sending chain

17.7 Analyzing the Signal Protocol

Putting everything together

Deniable

Choose $a \leftarrow [q]$
Compute $A = g^a$

Choose $b \leftarrow [q]$
Compute $B = g^b$

Fwd/back secure

Ephemeral
secret



A

B



Evolve
public key

Output B^a

Output A^b



$K = \text{KDF}(B^a)$



$K = \text{KDF}(A^b)$

AuthEnc is
deniable



$\text{AuthEnc}_K(P_1)$

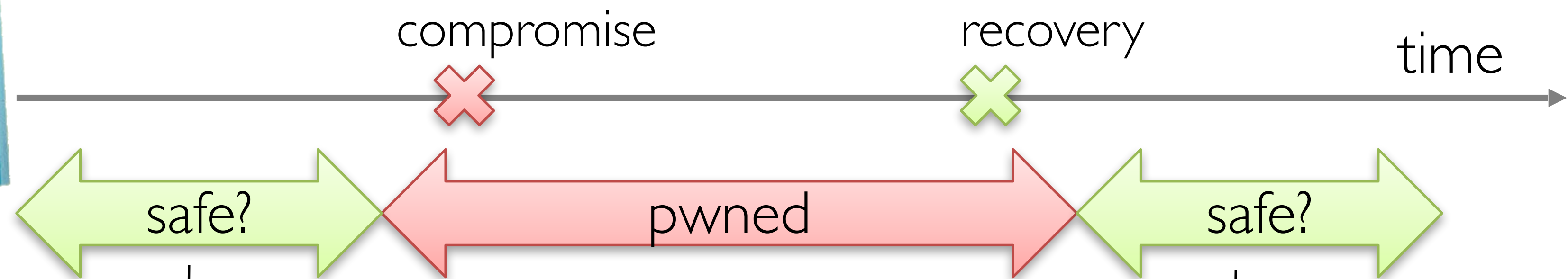
$\text{AuthEnc}_K(P_2)$

...



Evolve
symm key

Why Signal provides forward and backward secrecy



Forward (pre-compromise) secrecy

“The parties derive new keys for every Double Ratchet message so that earlier keys cannot be calculated from later ones.”

Backward (post-recovery) secrecy

“The parties also send Diffie-Hellman public values attached to their messages. The results of Diffie-Hellman calculations are mixed into the derived keys so that later keys cannot be calculated from earlier ones.”

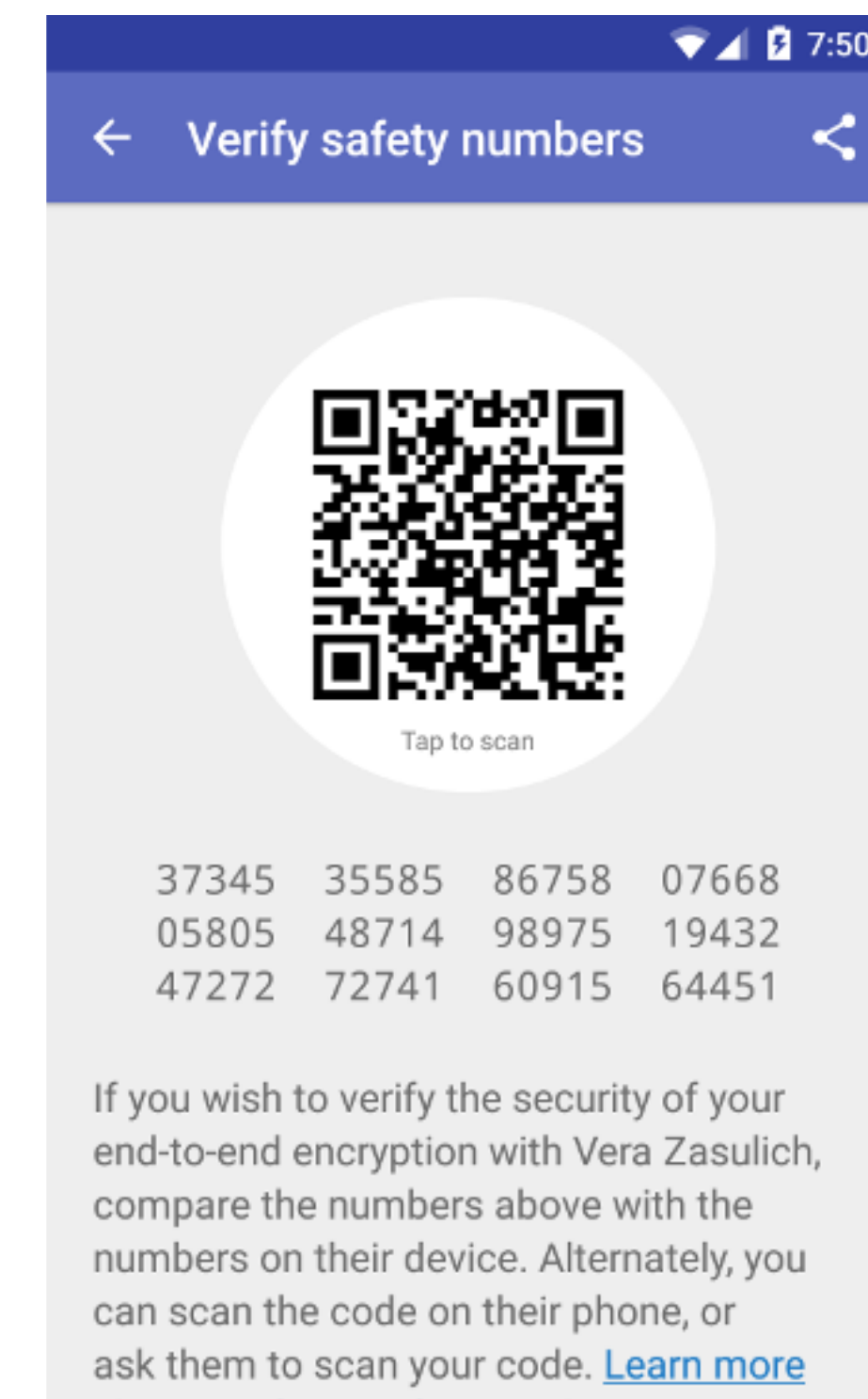
Ephemeral utopia

No long-term keys $\Rightarrow$ great forward secrecy

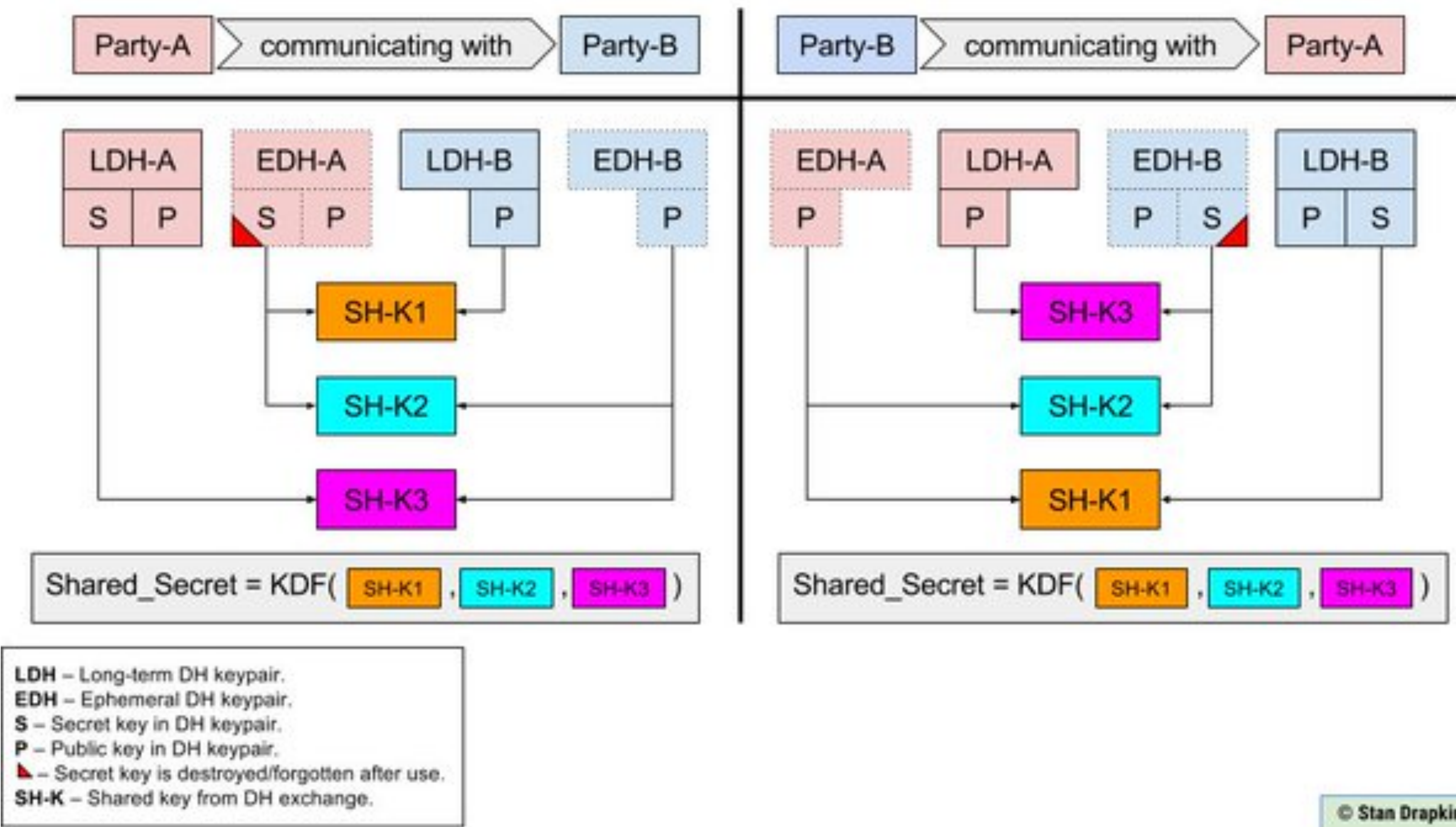
- Message key used to AuthEnc a message is used once and tossed
- Chain key used to construct msg key is refreshed in each public ratchet
- Diffie-Hellman key pairs chosen ephemeral in each public ratchet

Wait... actually, is this a utopia or a dystopia?

- If you don't have *any* long-term state, then who are you?!
- Resolution: Also have a long-term key, Signal maintains a PKI



Solution: a more involved *Triple-DH protocol*



SECURE MESSAGING APPS COMPARISON

BECAUSE PRIVACY MATTERS

App name	Allo	iMessage	Messenger	Signal	Skype	Telegram	Threema	Viber	Whatsapp	Wickr	Wire
TL;DR: Does the app secure my messages and attachments?	No	No	No	Yes	No	No	Yes	No	No	No	Yes

Source: <https://www.securemessagingapps.com>

Thinking About What You Need In A Secure Messenger

BY GENNIE GEBHART | MARCH 28, 2018



Source: <https://www.eff.org/deeplinks/2018/03/thinking-about-what-you-need-secure-messenger>

Roadmap of course so far

**Elegant
protocols**

**Utilitarian
tools**

